UML, Dependencies, Coupling

AND THE ANSWER TO LIFE AS A DEVELOPER

COMP3522 OBJECT ORIENTED PROGRAMMING 2: WEEK 3

UML REVIEW

We will use UML this term

Modelling language consisting of set of diagrams

Used to help developers to specify, visualize, construct, and document software system

1. Class diagrams
2. Collaboration diagrams
3. Sequence diagrams

Let’s review them

Class diagram

Purpose is to describe the static structure of the system

Shows the **classes**

- Attributes
- Operations

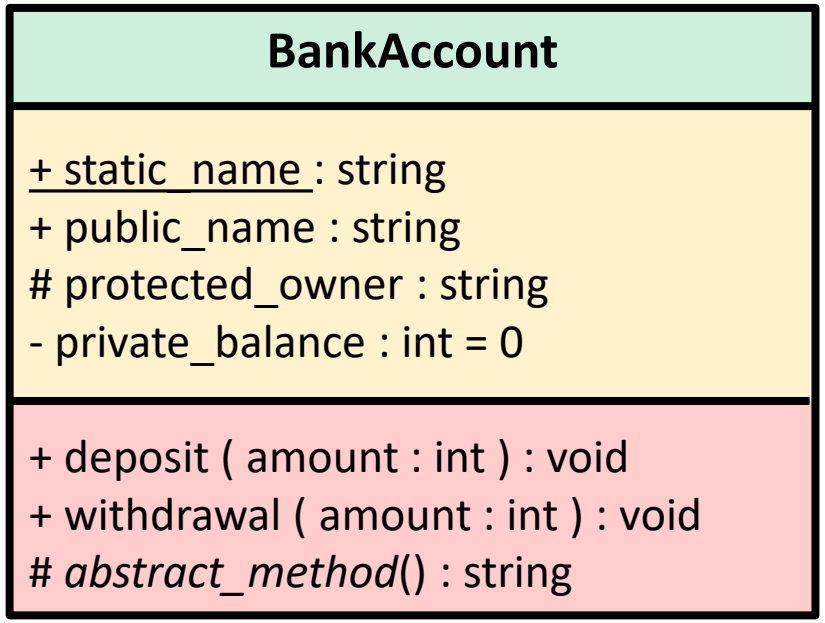
Relationships among objects

Language agnostic

Class diagram

Rectangle indicates a class

- **Top section is the name of the class**
- **Middle section are the attributes and types**
- **Bottom section are the methods**



Class diagram

Visibility

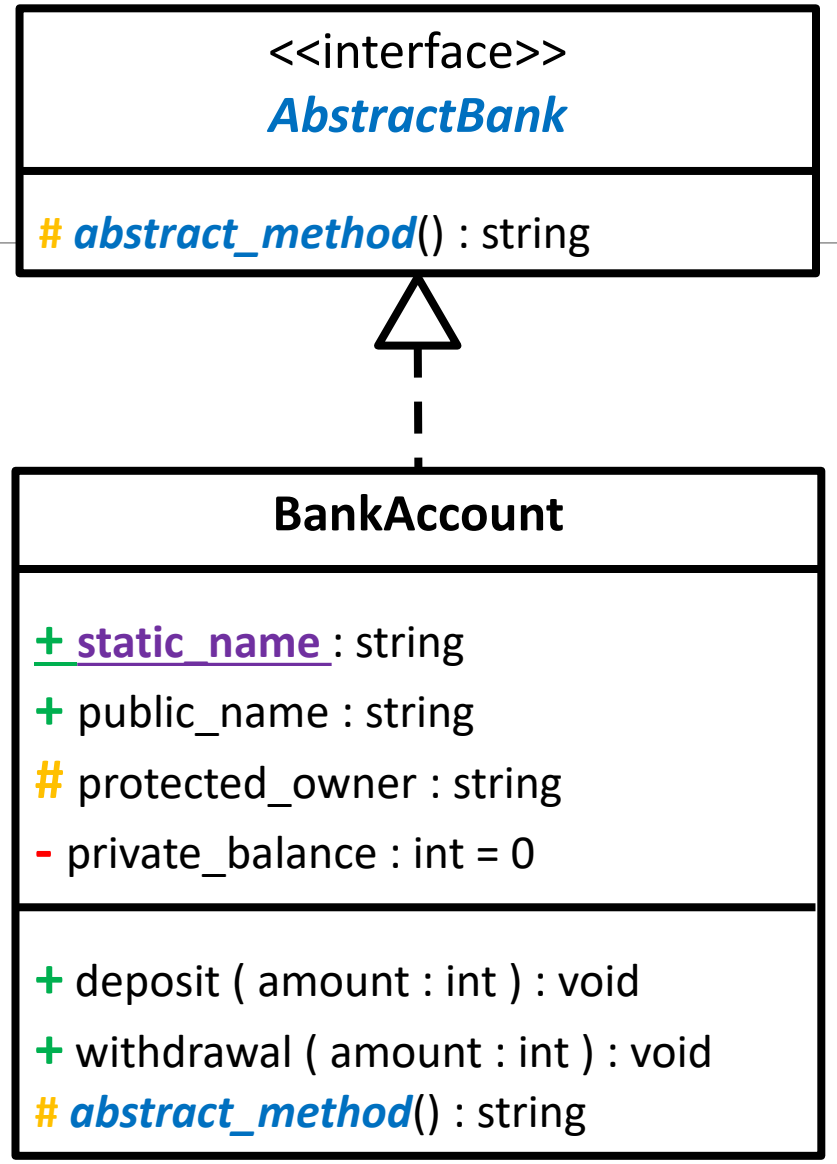
+ **public**

protected

- **private**

underline static attribute or method

italic abstract class or method



Class diagram – Relationships

Arrows indicating how two classes are related

Dependency – A contains code that temporarily **uses** B in some way. A **uses** B

A - - - -> B

```
class ClassA:
    def __init__(self):
        pass

    def a_method(self, class_b):
        class_b.b_method() # Dependency. Calls a method in ClassB

        return ClassB() # Dependency. Instantiates ClassB
                but doesn't save into self._attribute
```

- - ->

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram – Relationships

Arrows indicating how two classes are related

Association – A contains data attributes B. A **has-a** B. General form of aggregation and association

- Indicates **no conceptual** ownership, A does NOT own B.

- ie: Teacher → Student. Teacher teaches Students, but they don't conceptually own each other

A ⟶ B

```
class ClassA:
    def __init__(self, class_b):
        # Association
        # General term when ClassA saves ClassB as
            data attribute self._class_b or
            self._class_b_2
        self._class_b = ClassB()
        self._class_b_2 = class_b
```

→

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram – Relationships

Arrows indicating how two classes are related

Composition – A is composed of B. A **has-a** B, and if A is destroyed, B also destroyed.

- Indicates **strong conceptual** ownership, A strongly owns B

- ie: House ◆—— Room. House owns Rooms. If House destroyed, so are the Rooms

A ◆—— B

```
class ClassA:
    def __init__(self):
        self._class_b = ClassB() # Composition. Specific
            form of association when ClassB
            instantiated in scope AND saved in data
            attribute self._class_b
```

◆——

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram – Relationships

Arrows indicating how two classes are related

Aggregation – A is aggregated with B. A **has-a** B, and if A is destroyed B still exists.

- Indicates **weak conceptual** ownership, A weakly owns B

- ie: Library ◇—— Book. Library owns Books. If Library closes, Books can still exist

A ◇—— B

```
class ClassA:
    def __init__(self, class_b):
        self._class_b = class_b # Aggregation. Specific
            form of association when ClassB passed in
            from external source AND ClassB saved as
            data attribute self._class_b
```

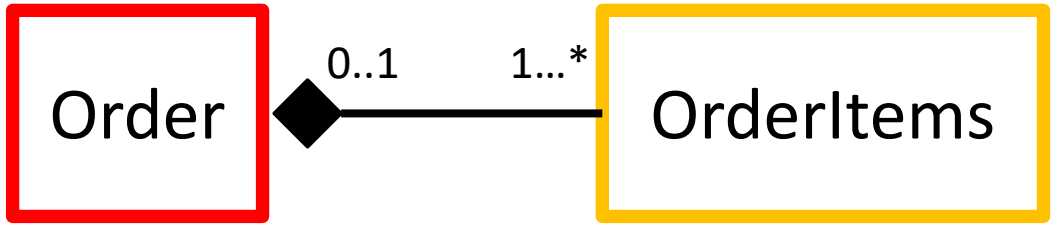
◇——

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram

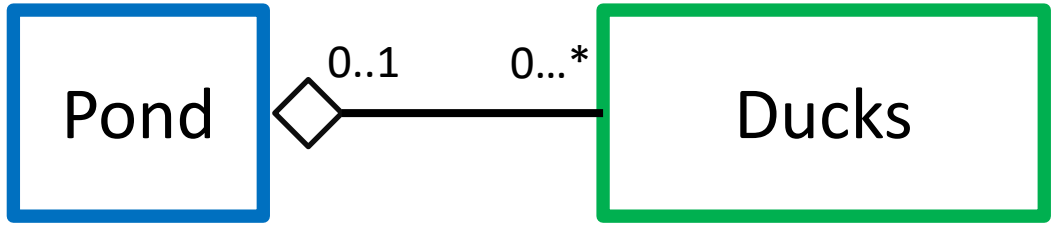
Composition: When a **class** is made up of and has references to **objects of another class**. In a composition relationship, if the **whole** is destroyed, the **parts** cannot exist.

Eg: If an ***Order*** is destroyed, then so are the ***OrderItems*** of which it is composed.



Aggregation: When a **class** references an **object of another class**. In an aggregation relationship, if the **class** referencing it is destroyed, the **referenced class** can live on.

Eg: If a ***Pond*** is destroyed, the **Ducks** can live on.



Class diagram – Relationships

Arrows indicating how two classes are related

Inheritance/Generalization – A is a subclass of B. A **is-a** B for **concrete** or **abstract** classes

A ⟶ B

```
class ClassA(ClassB): # Inheritance. ClassA inherits from ClassB
    pass
```

⟶

```
class ClassB:
    def __init__(self):
        self._attribute = “”
        pass

    def b_method(self):
        pass
```

Class diagram – Relationships

Arrows indicating how two classes are related

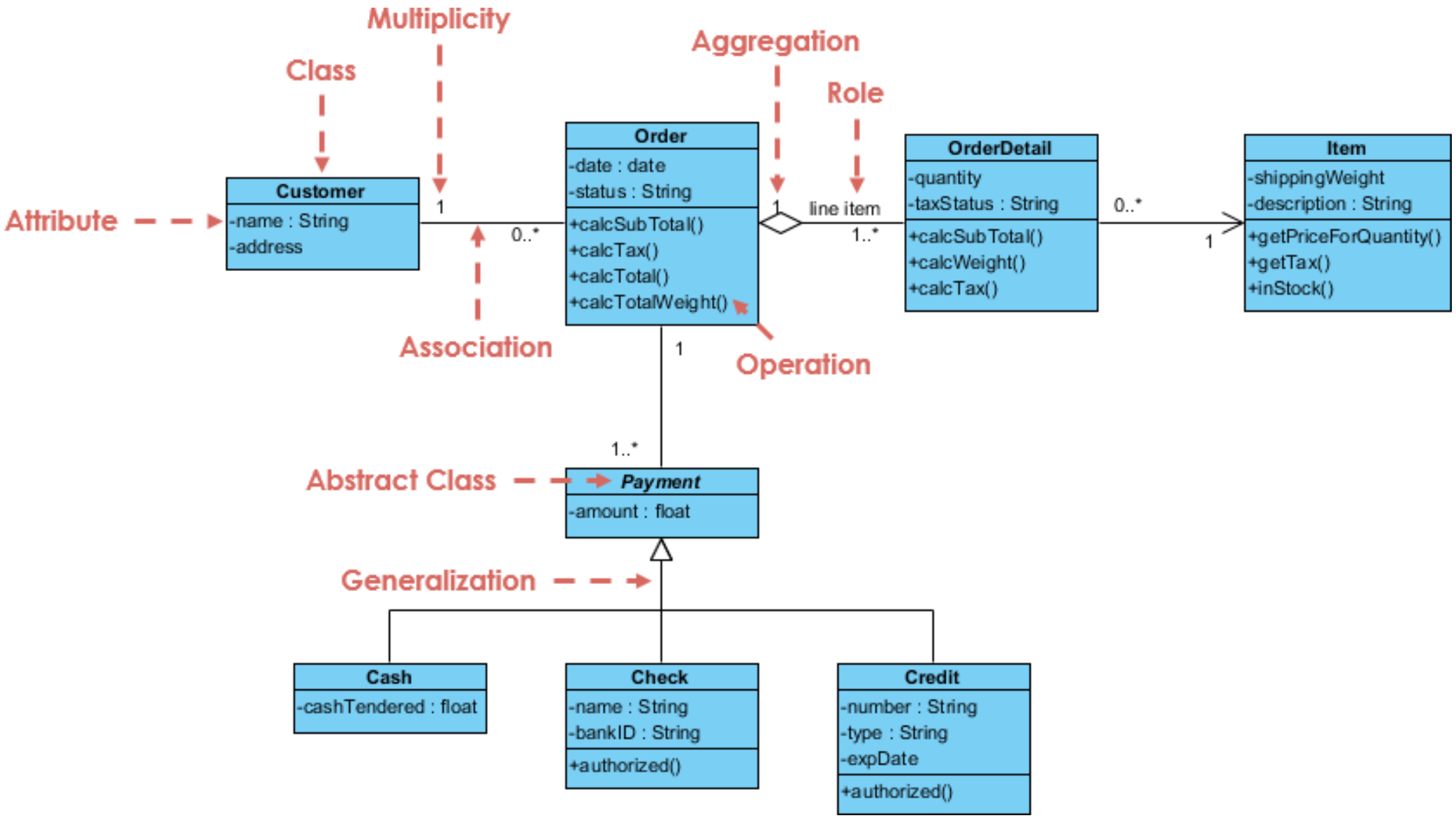
Realization/Implementation – A is implementing interface B. A **is-a** B for **interfaces**

A - - - ▷ B

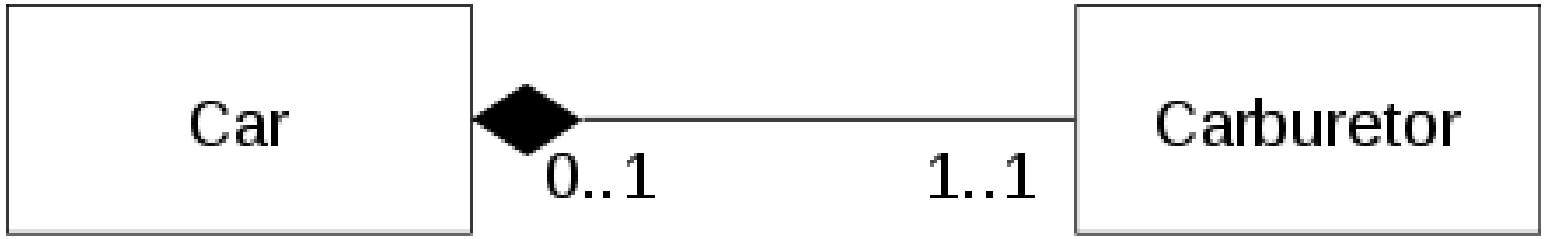
```
class ClassA(ClassB): # Realization. ClassA inherits from
                      ClassB and implements interface
                      ClassB's abstract methods
    def method(self):
        pass
```

- - - ▷

```
class ClassB(abc.ABC):

    #ClassB ONLY has
        abstract methods, no
        data attributes or
        implemented methods
    @abc.abstractmethod
    def method(self):
        pass
```

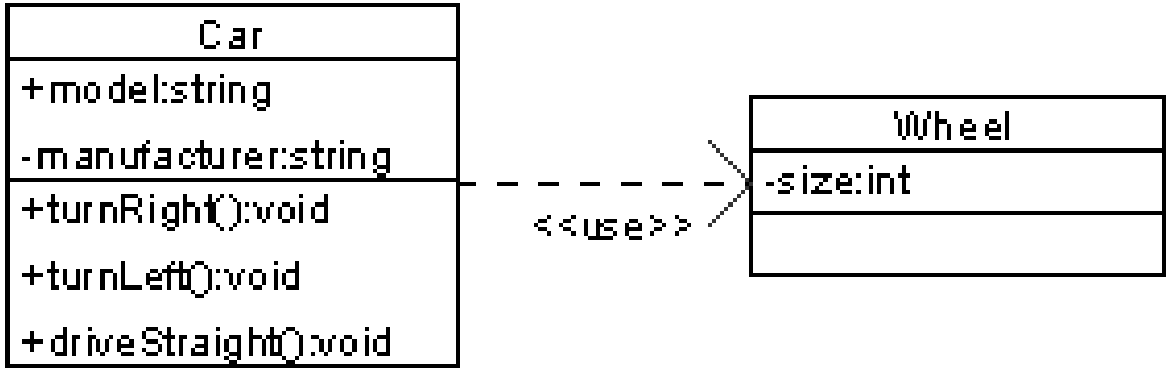

Multiplicity
Class
Aggregation
Role
Attribute
Association
Operation
Abstract Class
Generalization
Customer
-name : String
-address
Order
-date : date
-status : String
+calcSubTotal()
+calcTax()
+calcTotal()
+calcTotalWeight()
OrderDetail
-quantity
-taxStatus : String
+calcSubTotal()
+calcWeight()
+calcTax()
Item
-shippingWeight
-description : String
+getPriceForQuantity()
+getTax()
+inStock()
Payment
-amount : float
Cash
-cashTendered : float
Check
-name : String
-bankID : String
+authorized()
Credit
-number : String
-type : String
-expDate
+authorized()
1
0..*
1
line item
1..*
0..*
1
1
1..*

Class diagram



Car is composed of carburetors. Car has exactly 1 carburetor. Carburetor can belong to 0 to 1 cars

Class diagram

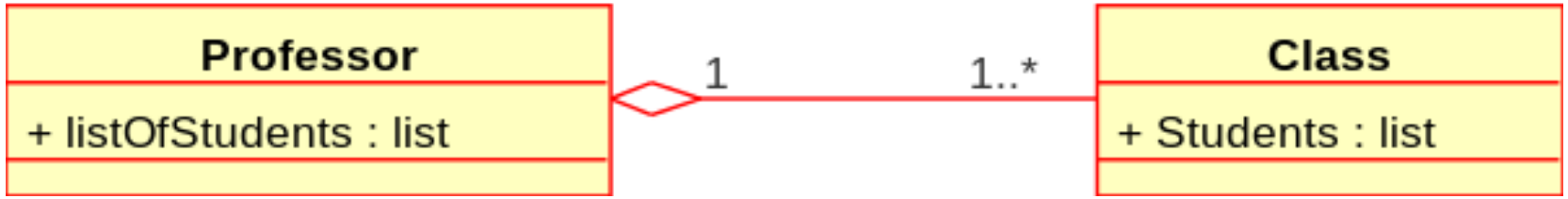


Car

- 2 strings named model, manufacturer
- Methods to turnRight, turnLeft, driveStraight

Car has a relationship with wheel. Some method uses wheel, perhaps accessing its size

Class diagram



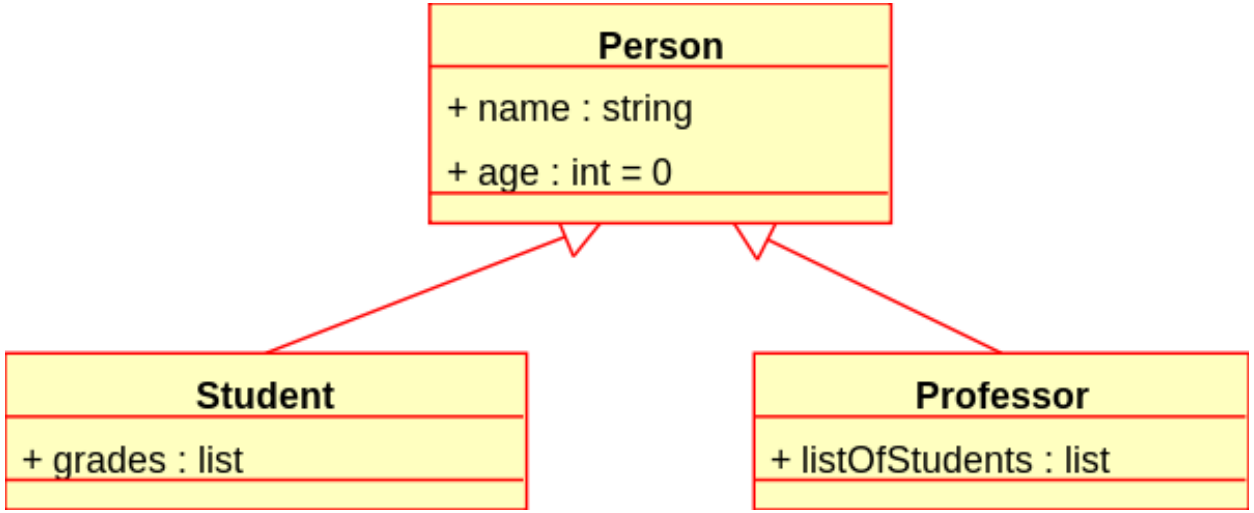
Professor is an aggregate of classes. Professor has 1 to many classes

- Has a list named listOfStudents

Each class has exactly 1 Professor

- Has a list named Students

Class diagram



Person class has a string name and int age as attributes

Student is a subclass of Person

- Has a list named grades

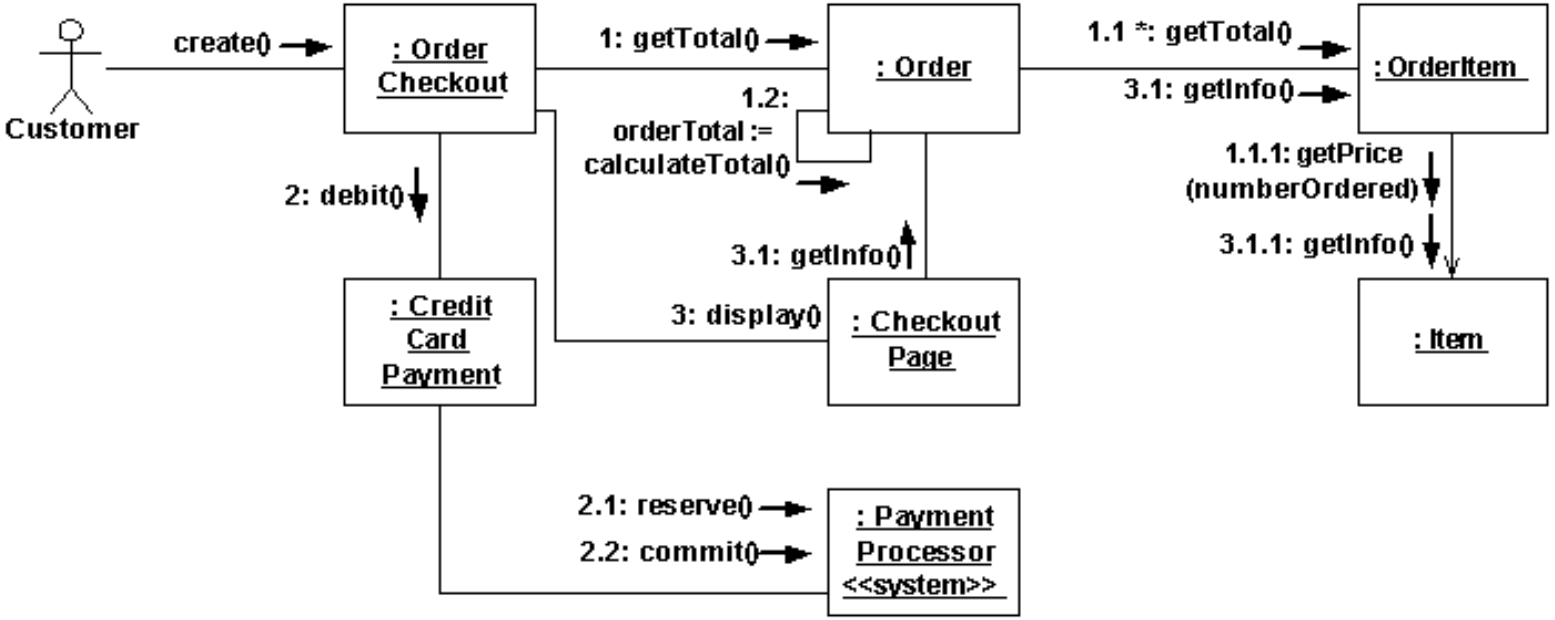
Professor is a subclass of Person

- Has a list named listOfStudents

Collaboration (communication) diagram

Presents interactions between objects as a sequence of messages between objects from a **structural perspective**

Shows how **objects interact** in a **particular use case** or subset of a use case



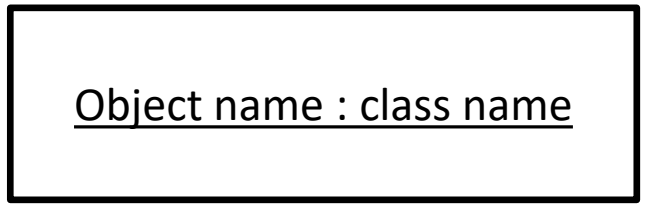
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml-collaboration-diagram/

Collaboration (communication) diagram

Notations (Object, Actor, Links, Messages)

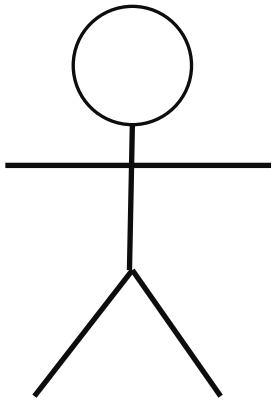
Object

- Object represented by a square containing the name of the object and its class underlined, separated by a colon



Actor

- Actor is the invoker of the interaction
- Actor is named and has a role



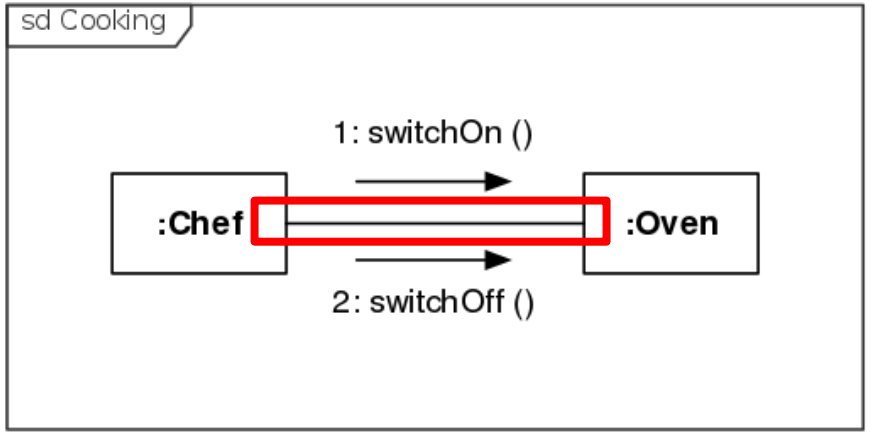
Customer

Collaboration (communication) diagram

Notations (Object, Actor, Links, Messages)

Links

- Connects objects and actors. They are instances of associations in the class diagram
- Messages are sent across these links
- Links are represented as a **solid line** between two objects

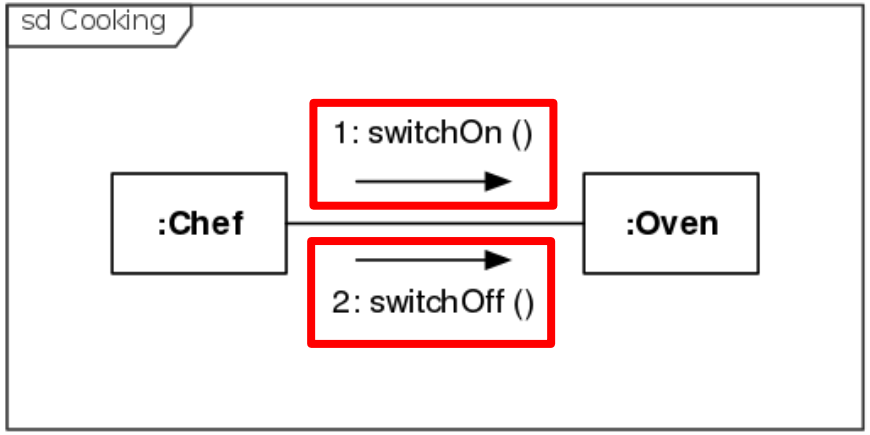


Collaboration (communication) diagram

Notations (Object, Actor, Links, Messages)

Messages

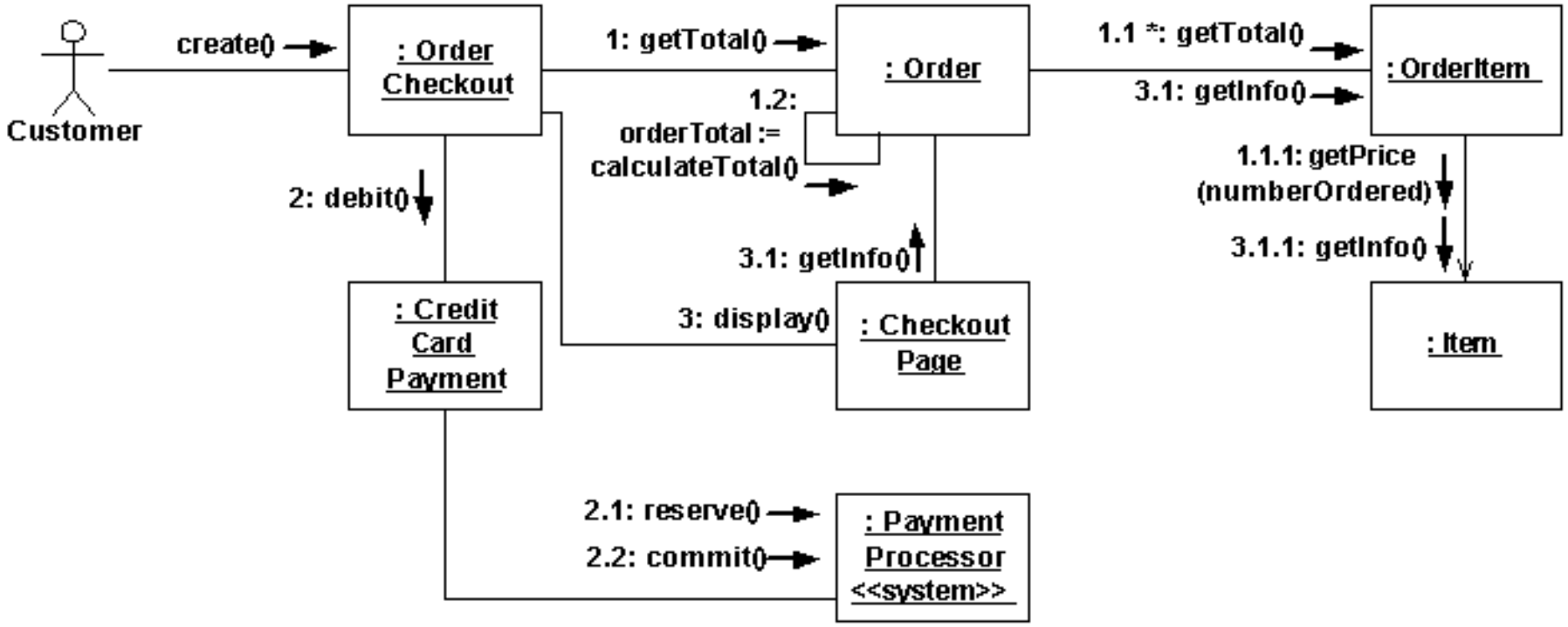
- Communication between objects that convey information with the expectation that activity occurs
- It is shown as a **labeled arrow** near a link
- Directed from sender to receiver
- Messages are numbered to indicate a sequence. Messages in the same call have additional decimals: 1.1, 1.2 etc



Collaboration (communication) diagram

Let's see this diagram again

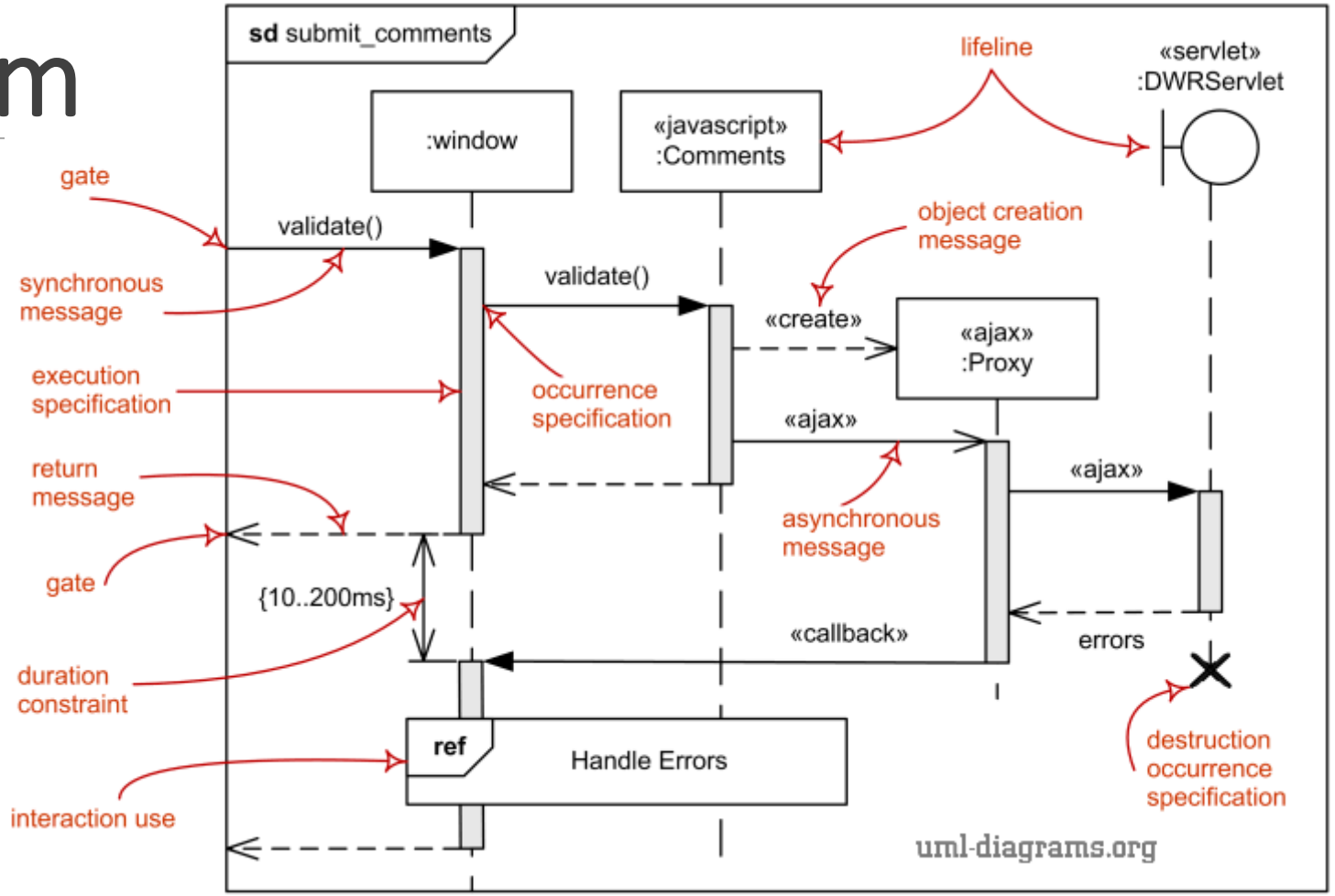
What use case is this collaboration diagram illustrating?



Sequence diagram

Presents **interactions between objects** as a sequence of messages between objects

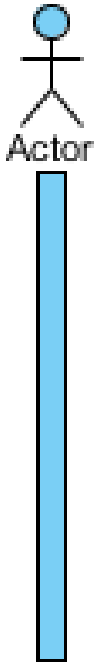
While a collaboration diagram is a structural approach, A sequence diagram is a **time-based perspective.** Includes the **lifetime** of the interactions between communicating objects



Sequence diagram

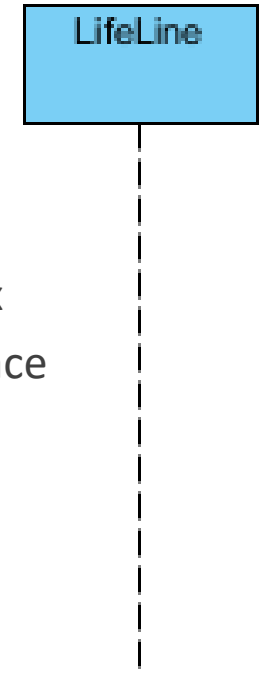
Actor

- Interacts with the subject
- External to the subject
- Played by human users, external hardware, other subjects



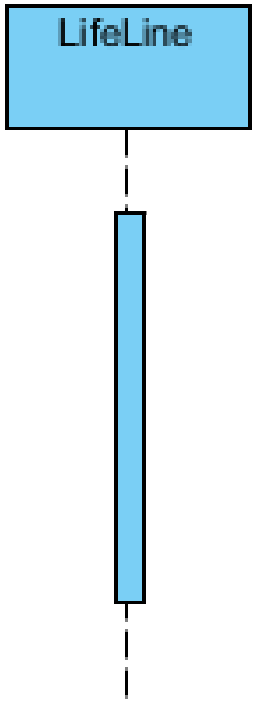
Lifeline

- Represents a single individual participant in the interaction
- Denoted by name and class type in the top box
- Dotted line indicates place in sequence



Activations

- Thin rectangle on the lifeline
- Represents the duration an element is performing an operation



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/

Sequence diagram

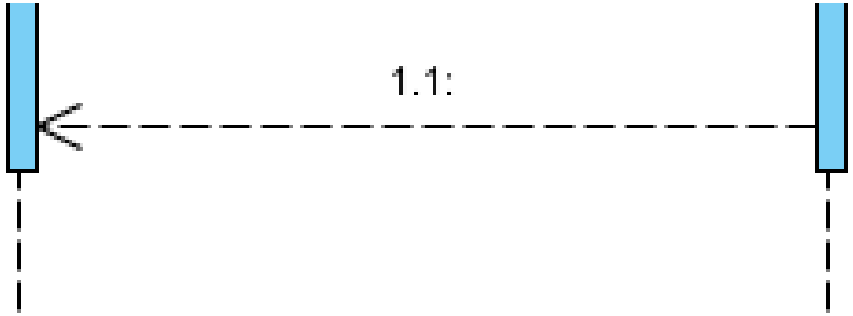
Messages: Indicates communication between lifelines of an interaction

Call message

- Represents invoking operation on the target lifeline
- Ie: Call a method on target lifeline

Return message

- Represents passing information back to the caller
- Ie: Return statement in method



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/

Sequence diagram

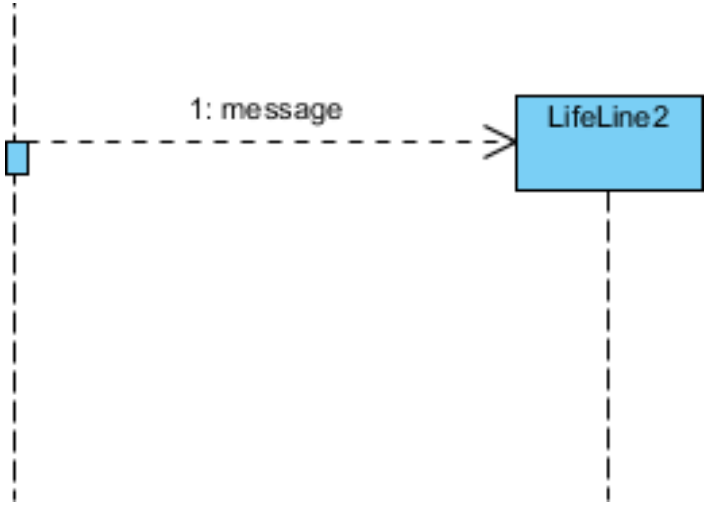
Messages: Indicates communication between lifelines of an interaction

Self message

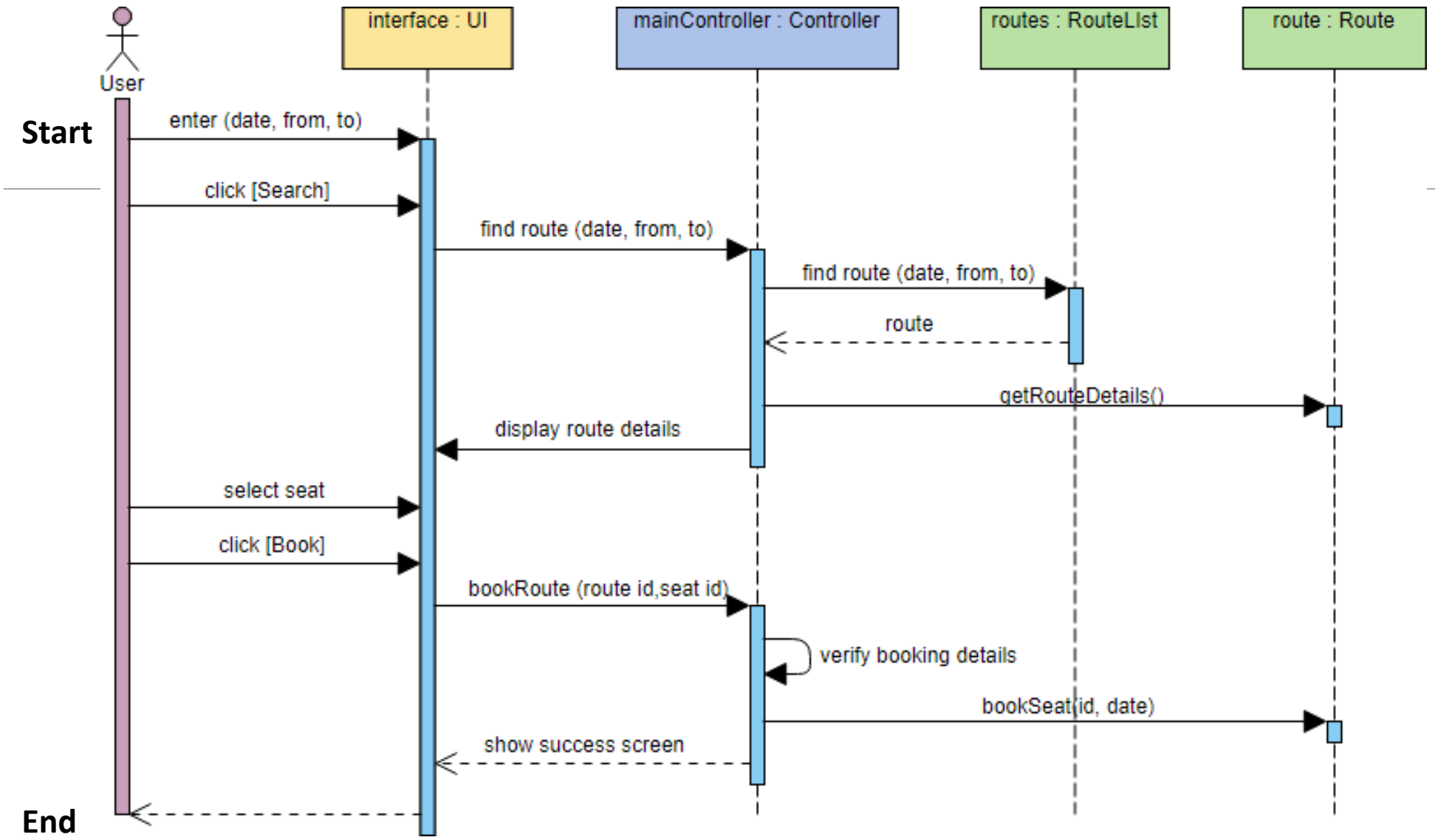
- Represents invoking message on the same lifeline

Create message

- Represents instantiation of the target lifeline



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/


User
interface : UI
mainController : Controller
routes : RouteLIst
route : Route
Start
enter (date, from, to)
click [Search]
find route (date, from, to)
find route (date, from, to)
route
getRouteDetails()
display route details
select seat
click [Book]
bookRoute (route id,seat id)
verify booking details
bookSeat(id, date)
show success screen
End

Dependencies : When your classes get clingy

Dependencies

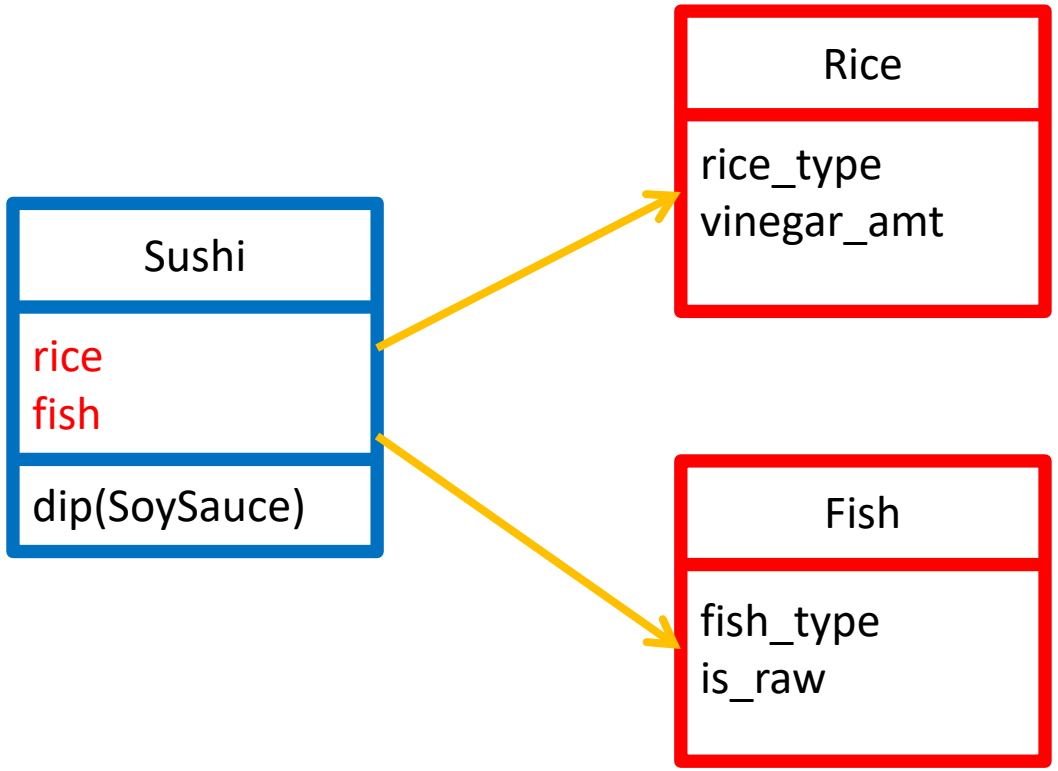
When one entity depends on another entity.

If entity A **uses** entity B, then A is said to be **dependent** on B.

In this context an entity could be a Module, Class, Package or Sub-system of related code

For example, say we were simulating a sushi restaurant. **Class Sushi would be dependent on Class Rice and Class Fish**

Why are dependencies bad?



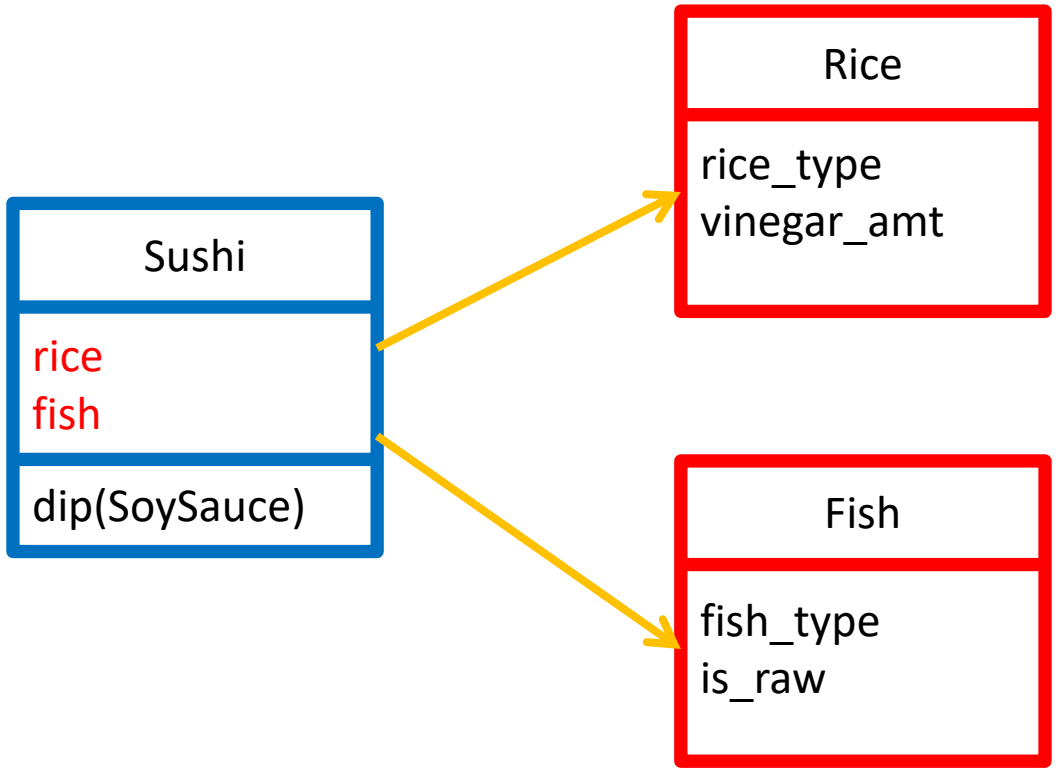
Dependencies on their own aren't "bad." Dependencies are required for entities in a system to communicate. There are certain types of dependencies that are bad

If entity A is **strongly dependent** on entity B then any change in entity B would possibly require us to change entity A as well.

This may not sound like much.

Imagine if A and B were sub-systems or packages in a larger application. Now this is an issue.

Why are dependencies bad?



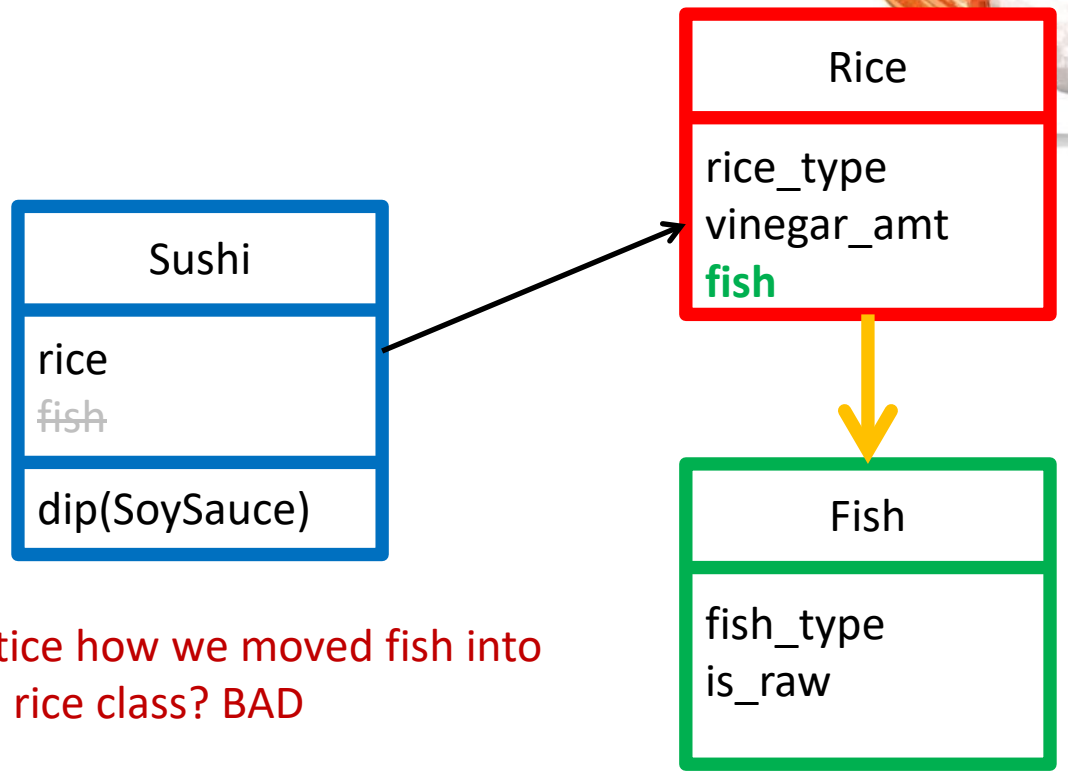
If entity A is **strongly dependent** with entity B then entity A will not be able to function without entity B.

Tight coupling is a form of **strong dependency**.

- The **dependency is so tight**, the system becomes **more rigid and less flexible**

We try to avoid tight coupling where possible

Tight coupling



Notice how we moved fish into the rice class? BAD

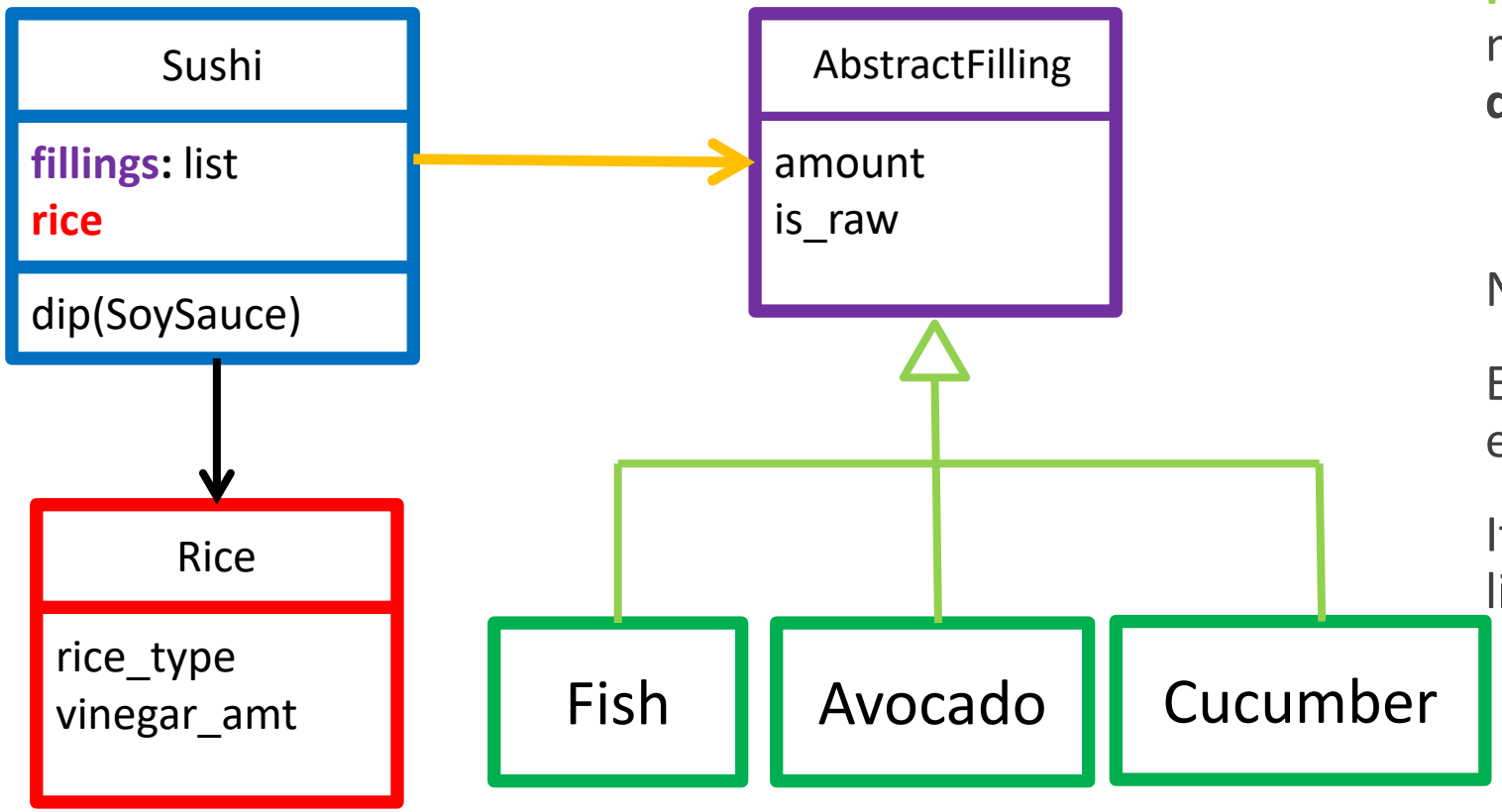
Say Rice and Fish are tightly coupled. This is bad because **Rice** will always come with **Fish**.

We wouldn't be able to have other kinds of sushi without fish.

- *(What about the Tamago!! :O)*

We don't want tight coupling like these. We want to **decouple our code** if possible so it can be flexible and polymorphic.

Decoupling sushi – Inheritance!



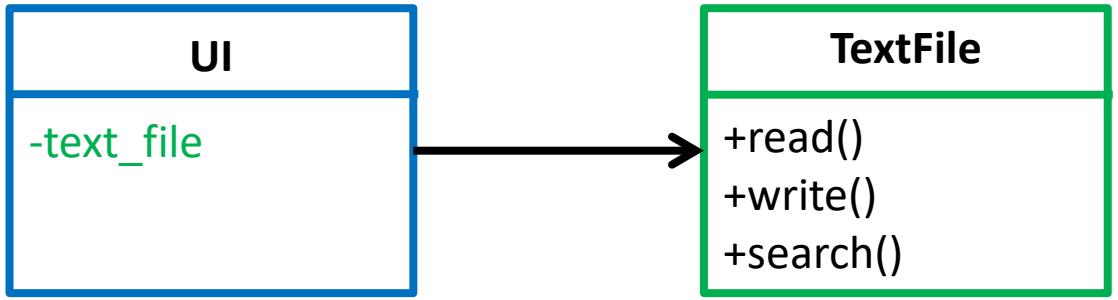
Inheritance and Interfaces are the most common and **effective way of decoupling code**.

NOTE: We still have a **dependency**!

Entities will always be dependent on each other, that's how OOP works

It's better to have **weak dependencies** like these

A more realistic example



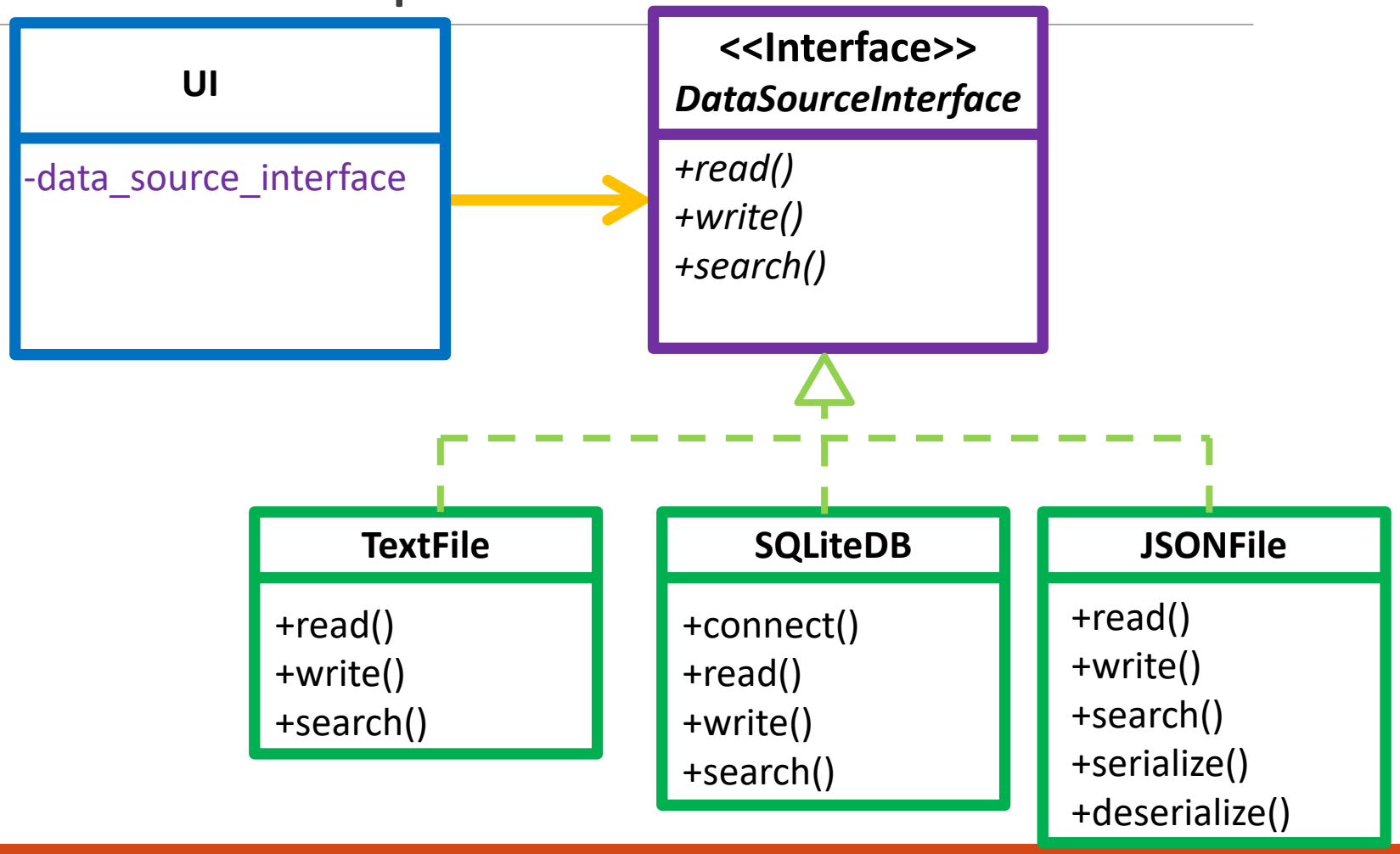
Say we were developing an app with a **UI** that was populated with some data from a **TextFile**.

After a few releases and years of development, for some reason we decide to switch out to a more secure source of data, perhaps an encrypted **JSON** file or even perhaps a **SQLite** Database. We would have to edit all the modules/classes that dealt with our app's **UI**!

A more realistic example

We can decouple our system to instead depend on a **DataSourceInterface** which is an **abstraction** that **hides** different **data sources**.

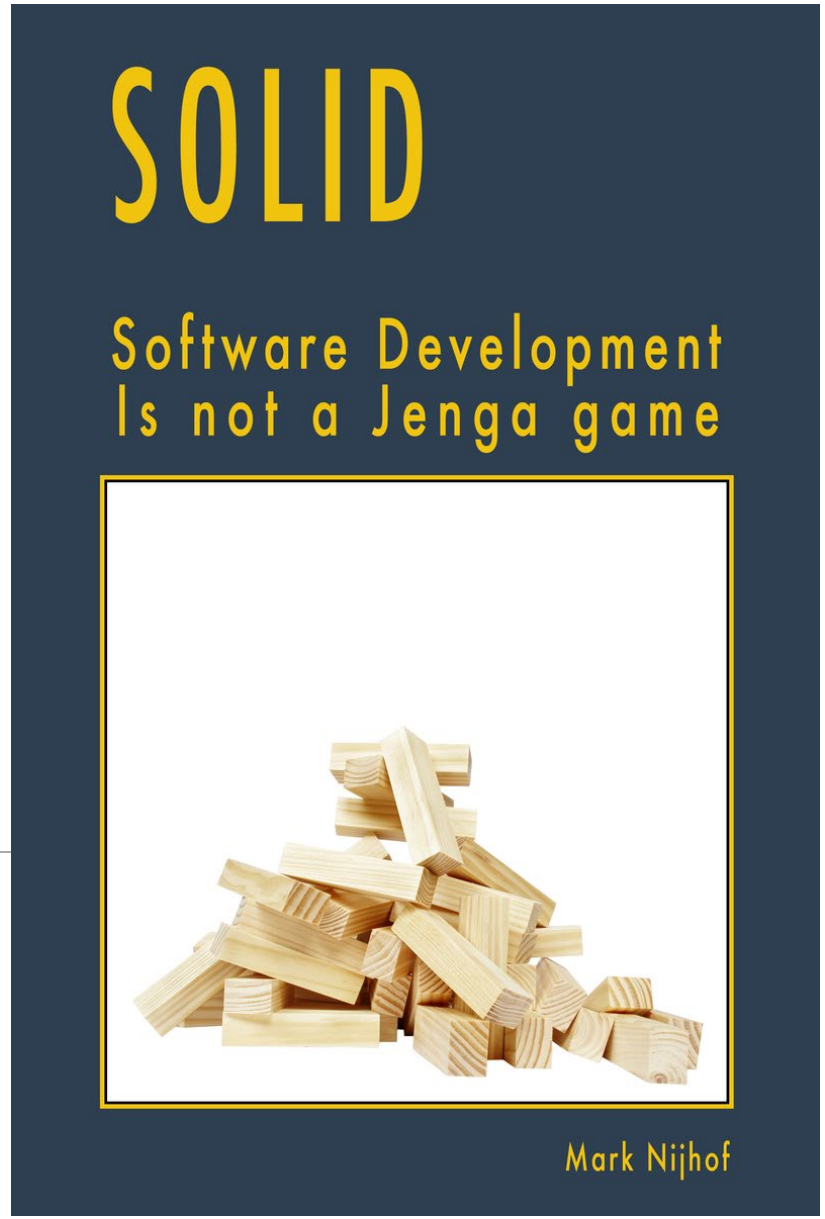
Our **UI** would just have to be **dependent on a common interface** provided by the **DataSourceInterface** and not be concerned with how that **data source** is implemented



SOLID Design Principles

AND THE LAW OF DEMETER.

PART 2



Recap: Dependencies & Coupling

Dependency:

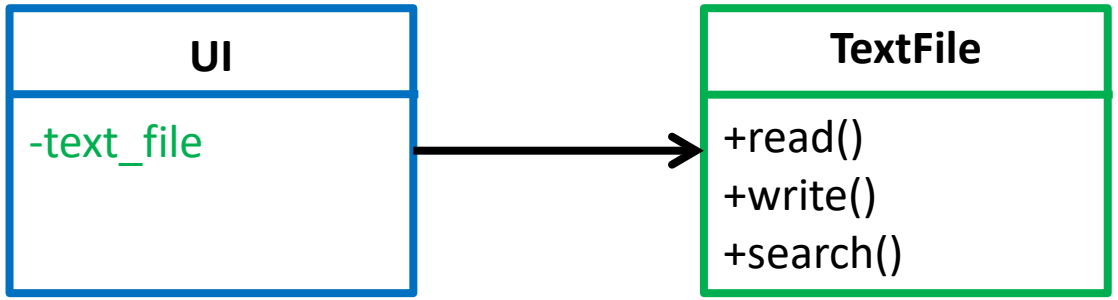
If **entity A uses entity B**, then **A** is said to be **dependent** on **B**.

Coupling:

If **entity A** is **tightly coupled** with **entity B** then **A** will not be able to function without entity **B**, **even if it was replaced** with another similar entity. **Entity A** would need to be refactored to use the new entity

We usually want to avoid this as it hinders us from creating flexible, dynamic, maintainable code.

Recap: Remember this?



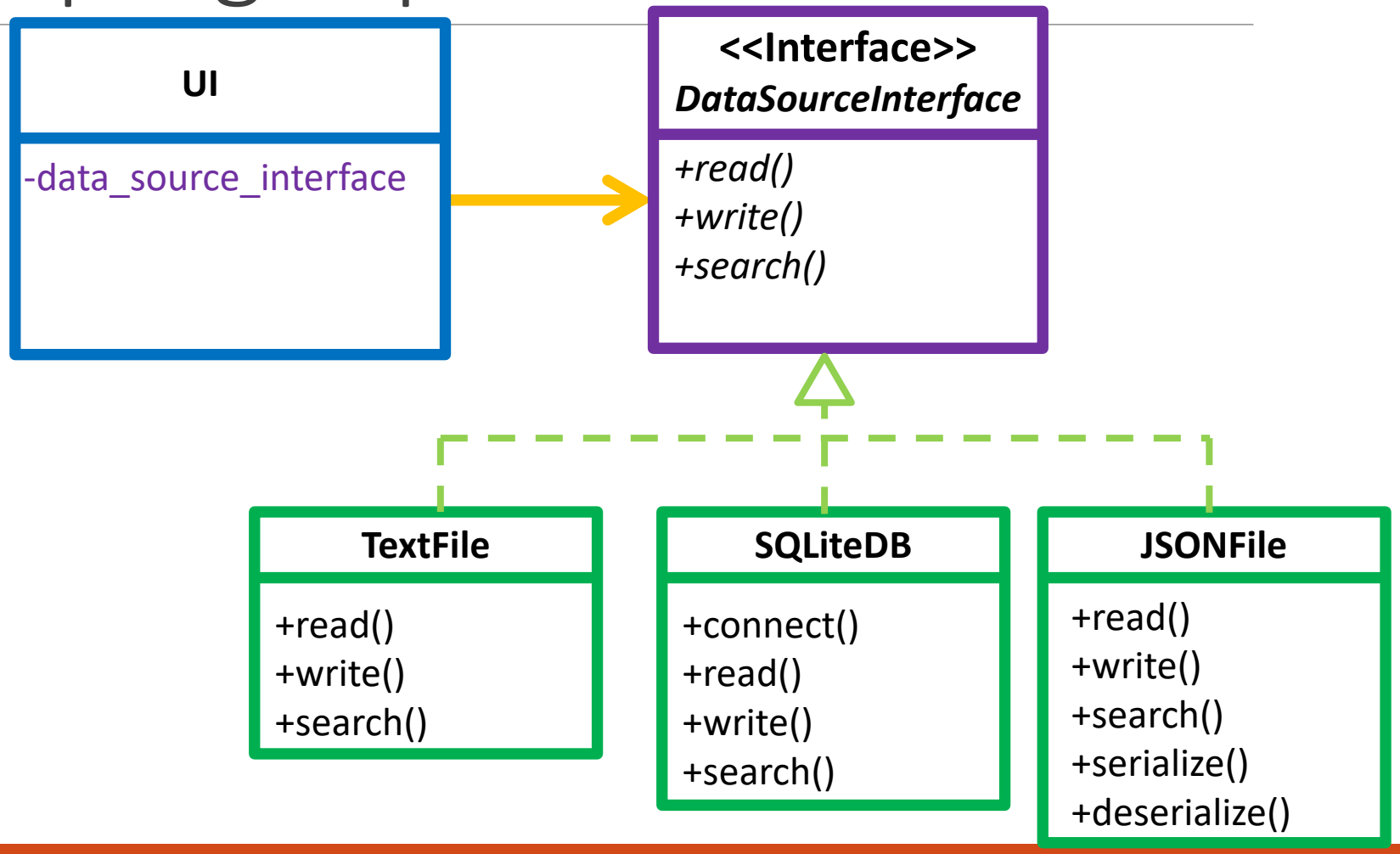
Say we were developing an app with a **UI** that was populated with some data from a **TextFile**.

After a few releases and years of development, for some reason we decide to switch out to a more secure source of data, perhaps an encrypted **JSON** file or even perhaps a **SQLite** Database. We would have to edit all the modules/classes that dealt with our app's **UI**!

Recap: Decoupling dependencies

We can decouple our system to instead depend on a **DataSourceInterface** which is an **abstraction** that **hides** different **data sources**.

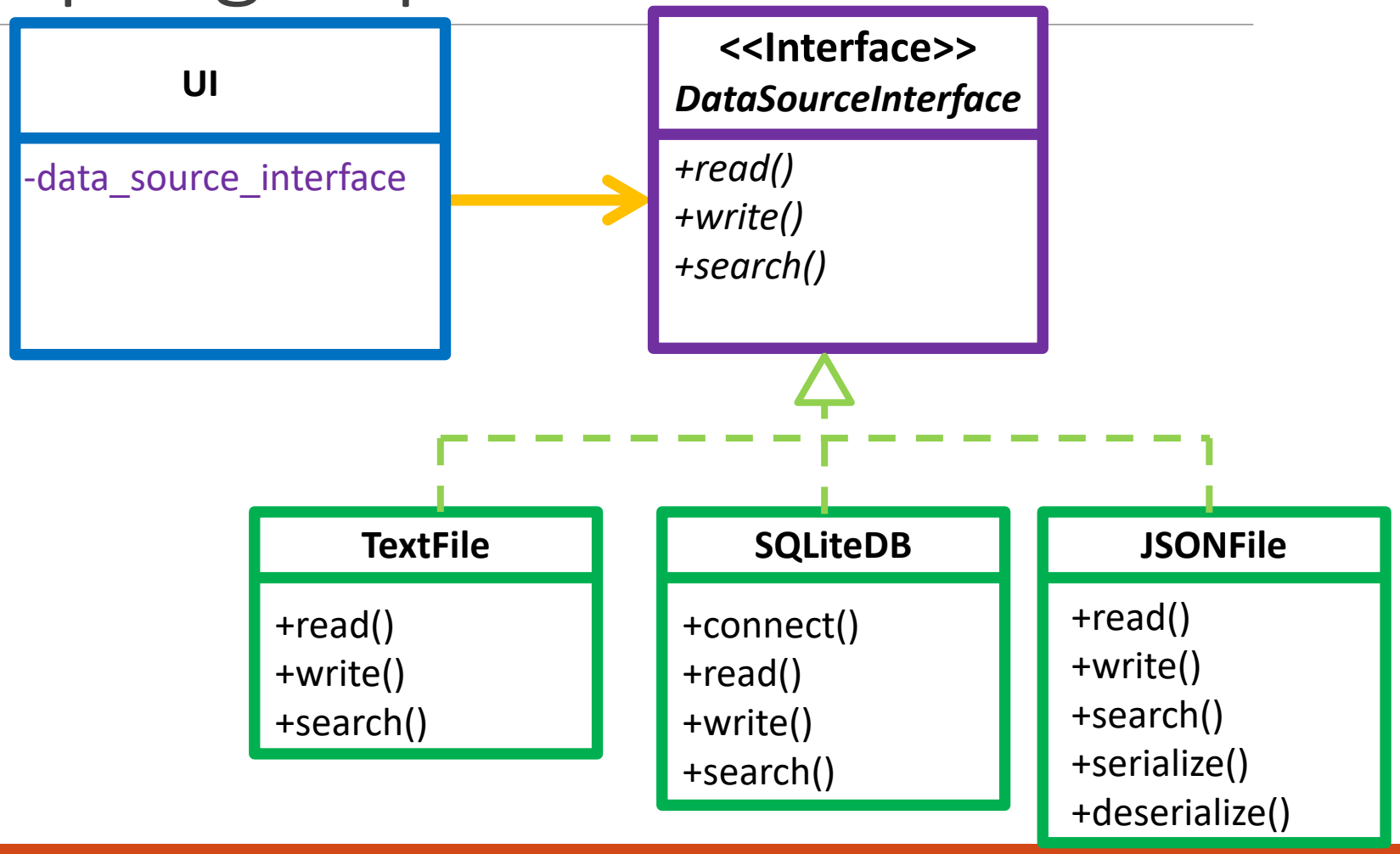
Our **UI** would just have to be **dependent on a common interface** provided by the **DataSourceInterface** and not be concerned with how that **data source** is implemented



Recap: Decoupling dependencies

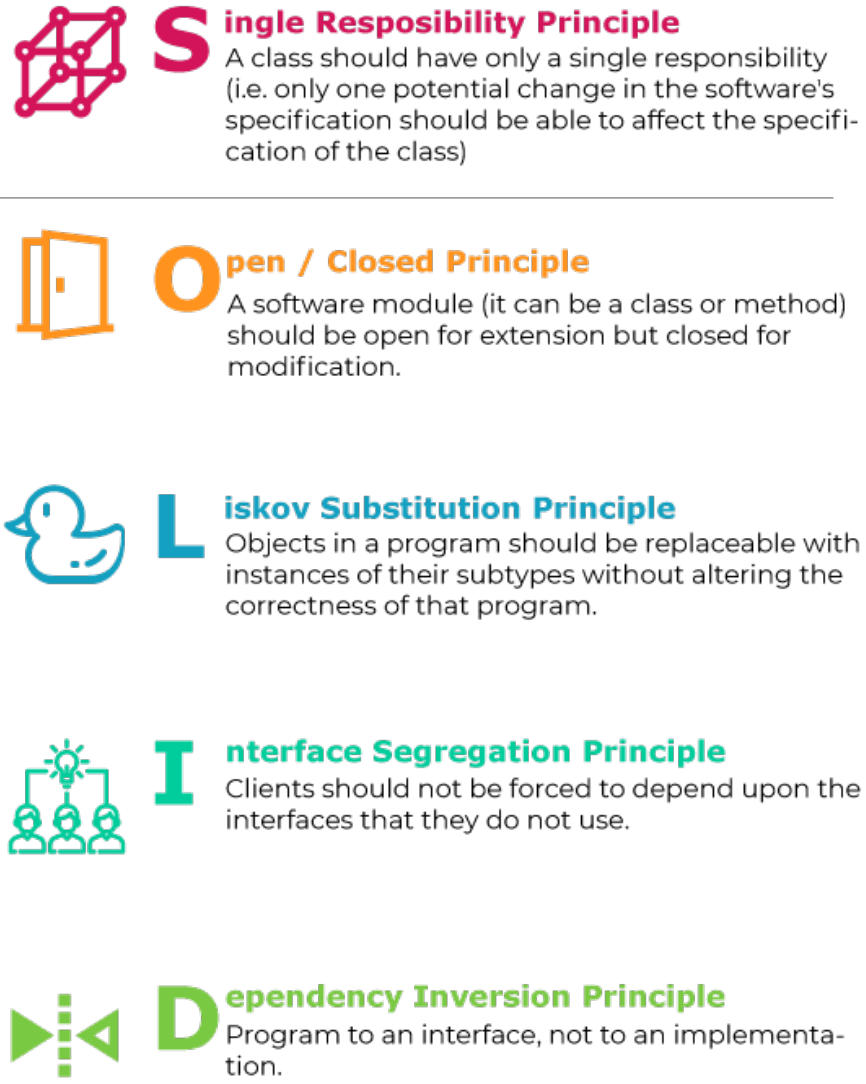
We actually used at least 3 **SOLID Design Principles** here!

Today we're going to learn what these principles are and why they are so important.



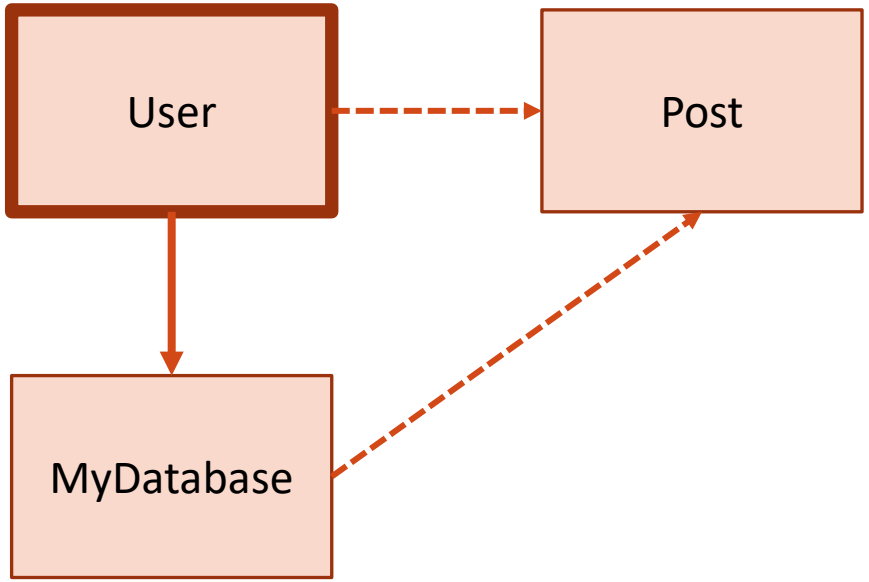
SOLID Design Principles

- A set of **guidelines** to help us write **maintainable**, **decoupled**, **flexible** Object Oriented Code
- Created by Robert C. Martin, also known as Uncle Bob amongst developers.
- Highly respected and has a bunch of books on writing clean code.



Single Responsibility Principle

Say we had a system where a user could write and create forum posts

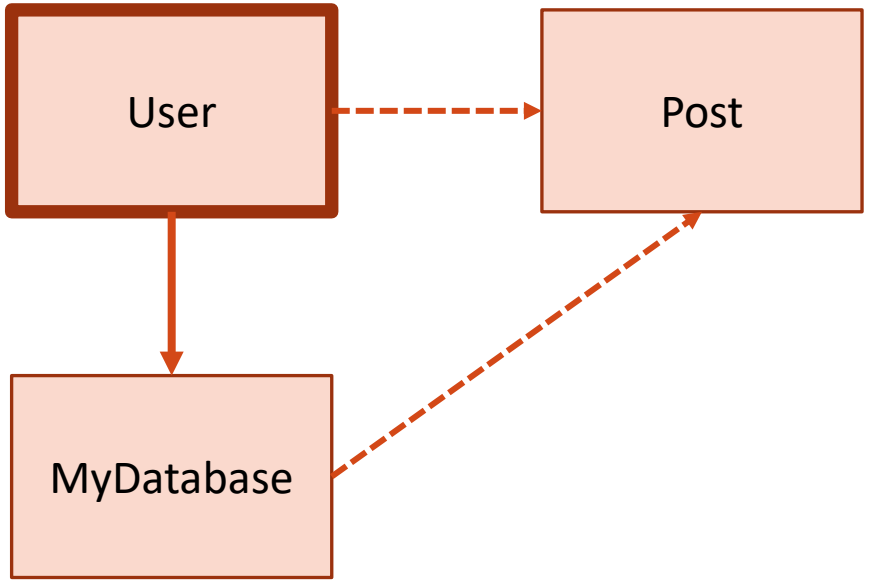



```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
        self._database = MyDatabase()

    def post_to_forum(message, subject):
        post = Post(subject, message, self.username)
        self._database.add_simple_post(post)
```

Why is this code bad?

The User class is very coupled and dependent. It carries out a number of **different roles** and **responsibilities**. Changing any part of this system (Post, MyDatabase) will probably cause a change in the User class as well.

Say we had a system where a user could write and create forum posts



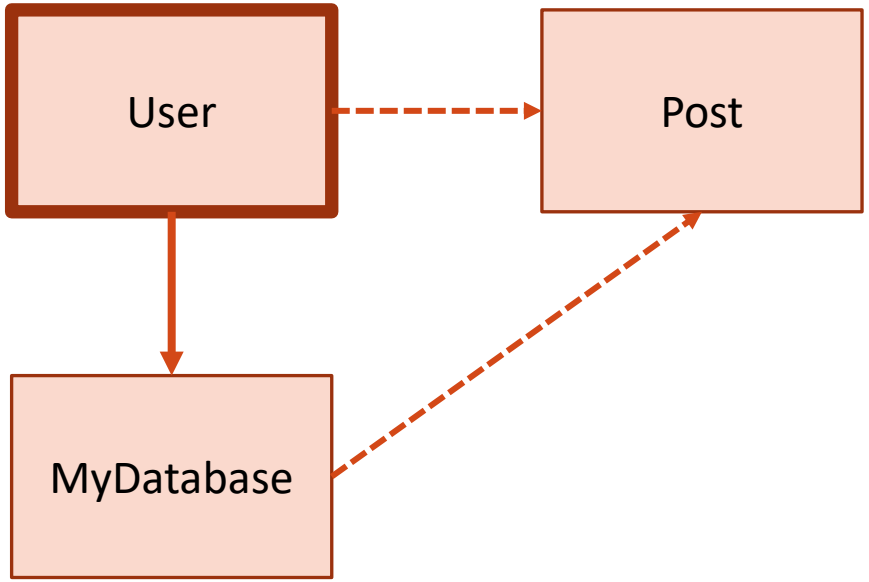

```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
        self._database = MyDatabase()

    def post_to_forum(message, subject):
        post = Post(subject, message, self.username)
        self._database.add_simple_post(post)
```

<u>Why is this code bad?</u>

User class has code to:

- **be a user**

Say we had a system where a user could write and create forum posts



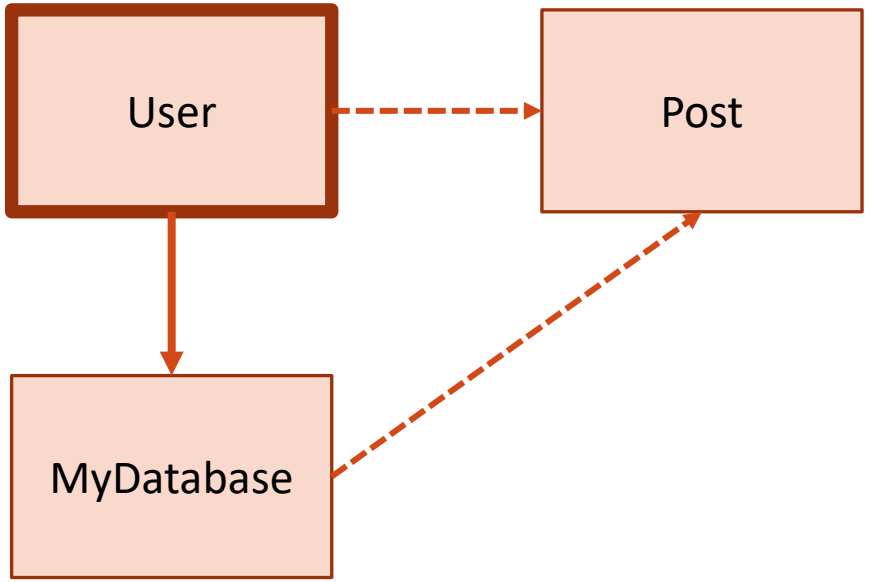

```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
        self._database = MyDatabase()

    def post_to_forum(message, subject):
        post = Post(subject, message, self.username)
        self._database.add_simple_post(post)
```

Why is this code bad?

User class has code to:

- be a user
- **create a database**

Say we had a system where a user could write and create forum posts



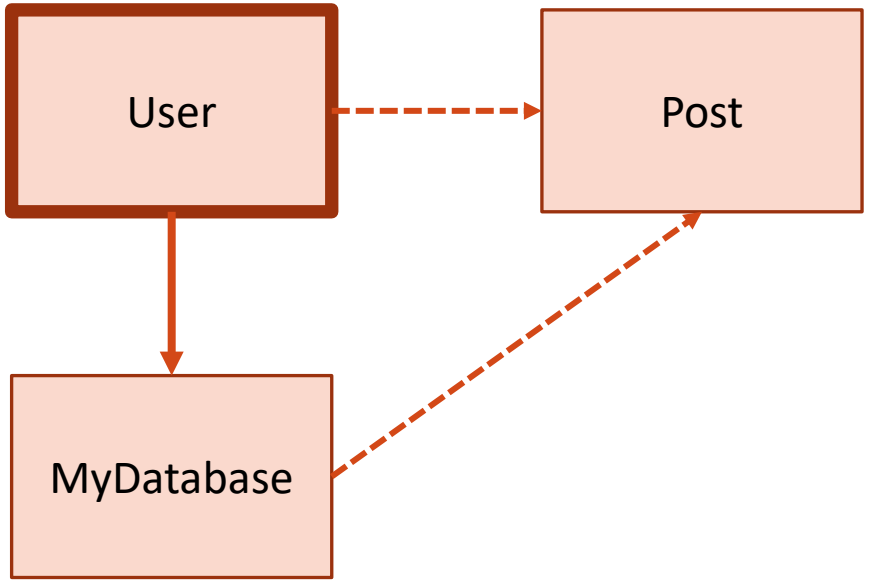

```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
        self._database = MyDatabase()

    def post_to_forum(message, subject):
        post = Post(subject, message, self.username)
        self._database.add_simple_post(post)
```

Why is this code bad?

User class has code to:

- be a user
- create a database
- **Post to a forum**

Say we had a system where a user could write and create forum posts



The **User class** should focus on only **being a user**

```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
        self._database = MyDatabase()

    def post_to_forum(message, subject):
        post = Post(subject, message, self.username)
        self._database.add_simple_post(post)
```

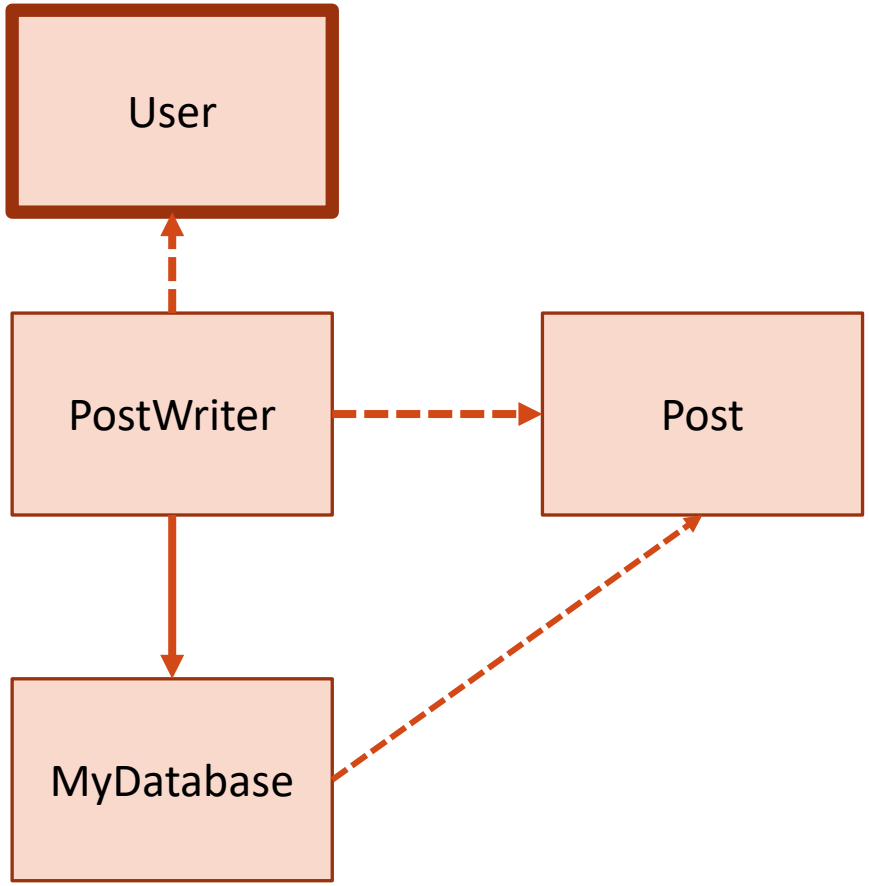
Single Responsibility Principle

"The Single Responsibility Principle requires that each class is responsible for only one thing."

Ask yourself, are there multiple reasons to change my class?

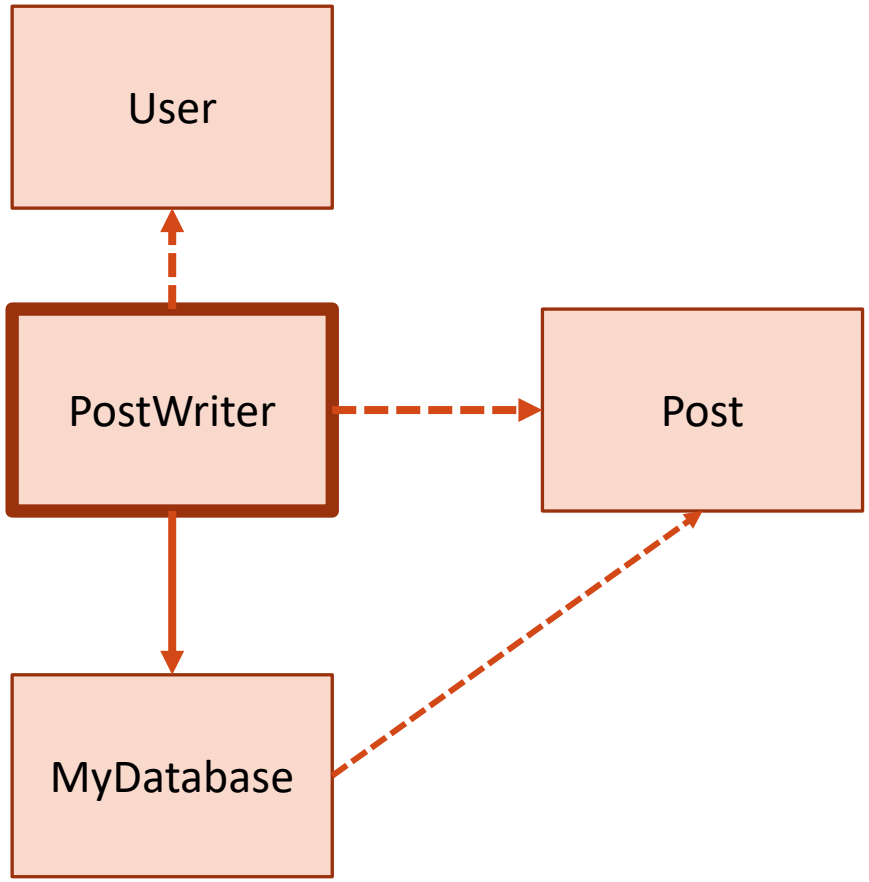
There should only be one reason to change your class. Each class should have one and only one responsibility.

We Split the User Class! Pt.1




```
class User:
    def __init__(self, username, password, name, email):
        self._username = username
        self._password = password
        self._name = name
        self._email = email
```

We Split the User Class! Pt.2




```
class PostWriter:
    def __init__(self, database):
        self._database = database

    def post_to_forum(self, user, message, subject):
        post = Post(subject, message, user._username)
        self._database.add_post(post)
```

Open-closed principle

Now, Say we had a UI widget class….

```
class UIWidget:

    def draw(self):
        if self._type == "Button":
            # complicated button draw logic here
        else:
            # complicated draw logic for other widget types
```

UIWidget
- type : string
+draw()

Say we had a graphical user interface, where each UI element was a **UIWidget**
It works great initially, but eventually we want to add more widget types…

Now over time, what if our UIWidget class looked like this?

```
class UIWidget:
    def draw(self):
        if self._type == "Button":
            #complicated button draw logic here
        elif self._type == “TextBox”:
            # complicated draw logic for a generic UI box
        elif self._type == “ScrollBar”:
            # complicated draw logic for a generic UI box
        elif self._type == “Label”:
            # complicated draw logic for a generic UI box
        else:
            # complicated draw logic for a generic UI box
```

UIWidget
- type : string
+draw()

This is **terrible design**! If we wanted to change the logic / algorithm behind drawing a button, we would need to edit this massive draw method.

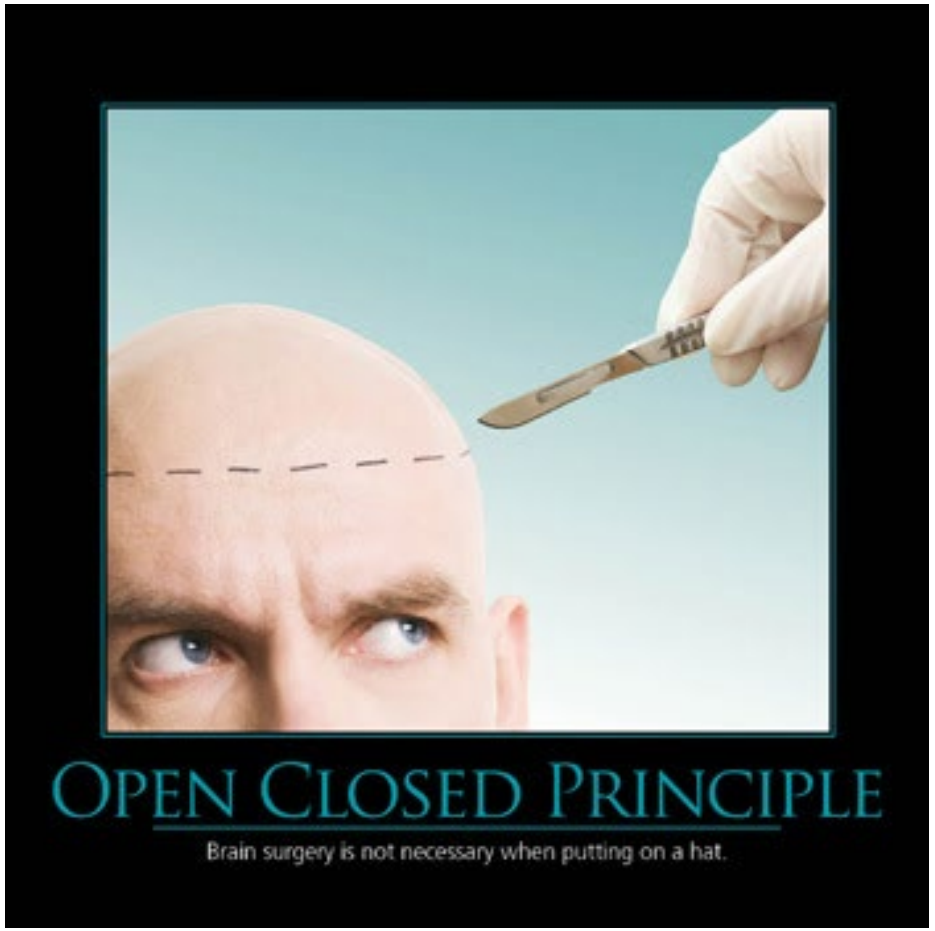
An error or bug here would affect all the widgets on screen and crash our whole system!

Open Closed Principle

The Open/Closed Principle states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

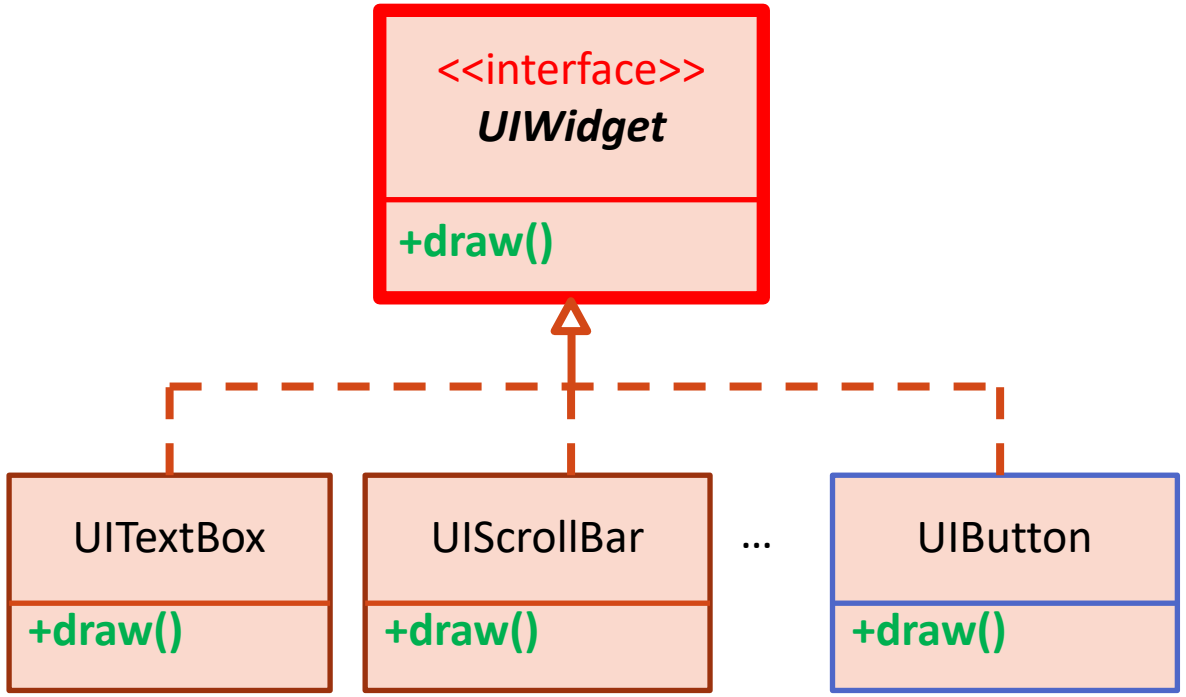
Once a class is made, it's behaviour shouldn't be modified to support special cases. It should be **CLOSED for modification**.

Instead, we extend (or inherit) from the class and override behaviours. That is, it's **OPEN for extension**.



We use Inheritance (and overriding)!

```
class UIWidget(abc.ABC):
    @abc.abstractmethod
    def draw(self):
        pass

class UIButton(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UITextBox(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UIScrollBar(UIWidget):
    def draw(self):
        # complicated draw code for scroll bar goes here
```

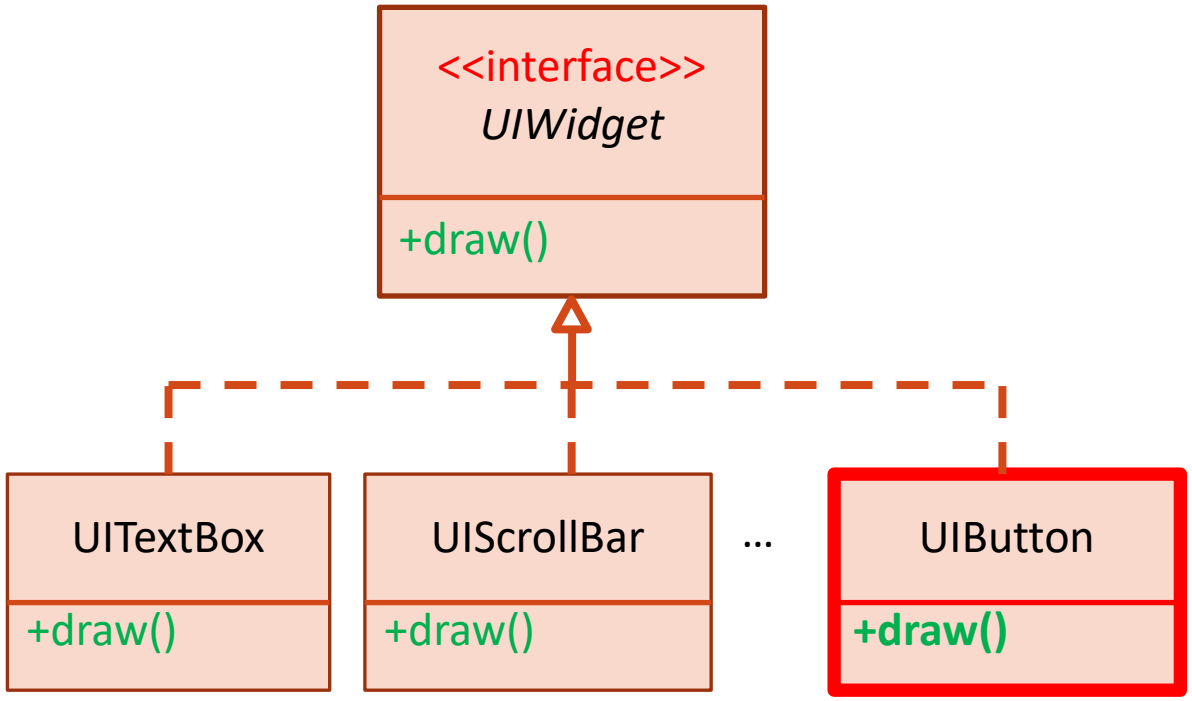


We used abstraction to do this.

The **UIWidget class** is now an **interface** ensuring that each UI component that inherits from it has the **same interface**.

We use Inheritance (and overriding)!

```
class UIWidget(abc.ABC):
    @abc.abstractmethod
    def draw(self):
        pass

class UIButton(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UITextBox(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UIScrollBar(UIWidget):
    def draw(self):
        # complicated draw code for scroll bar goes here
```



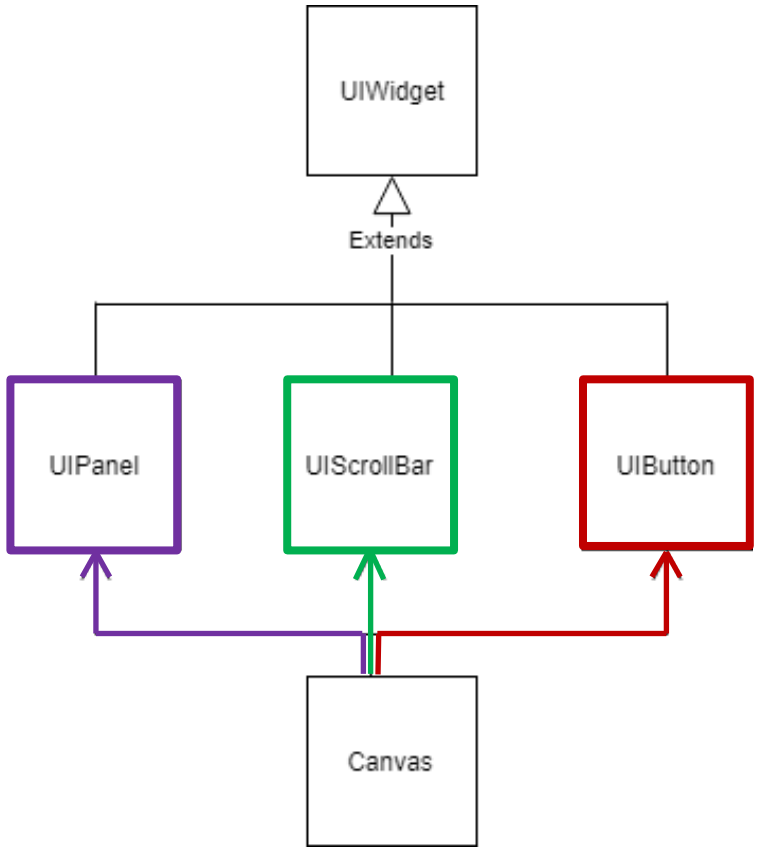
Now, any change to **UIButton** would not affect any other widget.

Each widget class is modular and decoupled. It even follows the Single Responsibility Principle!

Liskov Substition Principle

Now how do we draw all these widgets?

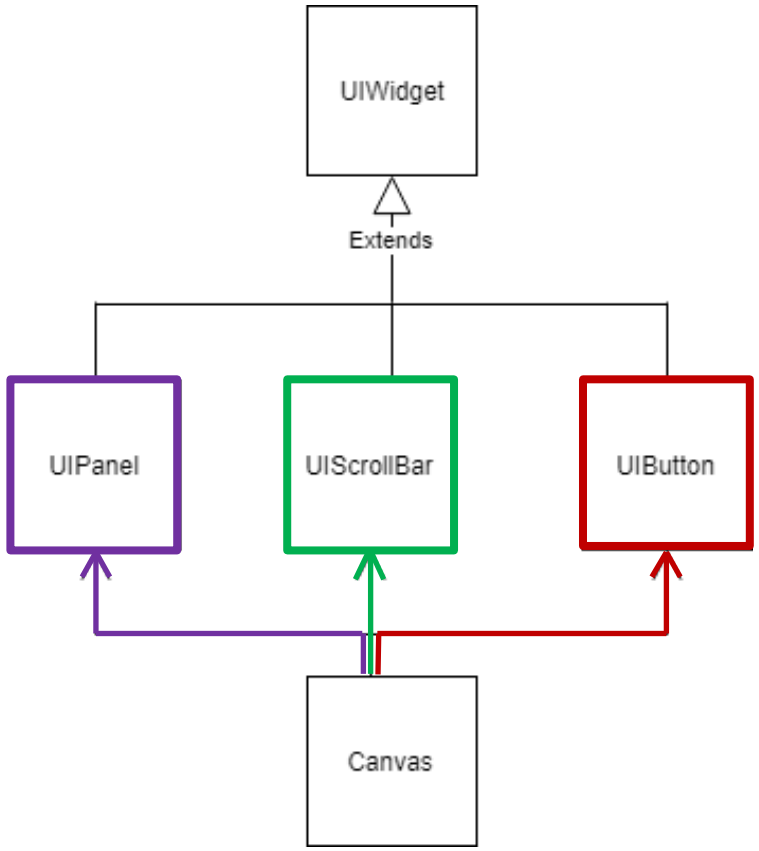
```
class Canvas:
    def __init__(self, panel, scrollbar, …, button):
        self._panel = panel
        self._scrollbar = scrollbar
        self._button = button
        ..
    def draw_screen(self):
        self._panel.draw()
        self._scrollbar.draw()
        self._button.draw()
        ..
        ..
```



Say we have a **Canvas** object that needs to draw the screen. Would this be a good way of drawing all the widgets?

Now how do we draw all these widgets?

```
class Canvas:
    def __init__(self, panel, scrollbar, …, button):
        self._panel = panel
        self._scrollbar = scrollbar
        self._button = button
        ..
    def draw_screen(self):
        self._panel.draw()
        self._scrollbar.draw()
        self._button.draw()
        ..
        ..
```



This would be terrible! Why inherit if you have to keep an**explicit reference** to each subtype?

Liskov Substitution Principle

The **Liskov substitution principle** states that if ***S*** is a subtype of ***T***, then objects of type ***T*** may be replaced (or substituted) with objects of type ***S***.

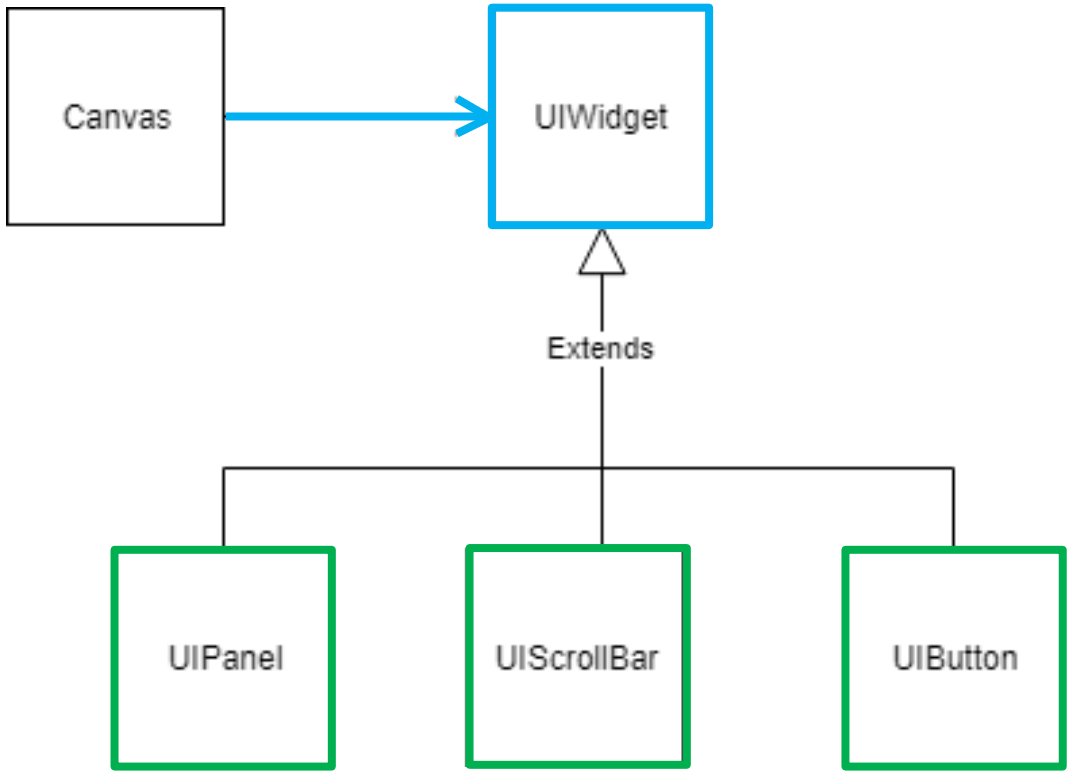
OR

Objects in a program should be replaceable with instances of their **base types** without altering the correctness of that program.

Simply put, you should be able to refer to all the different **child UI** components as its parent class, **UIWidget**

In Python this is easy, since we don't worry about types!

```
class Canvas:
    def __init__(self, widget_list : UIWidget):
        self._ui_widgets = widget_list

    def draw_screen(self):
        for widget : UIWidget in self._ui_widgets:
            widget.draw()
```



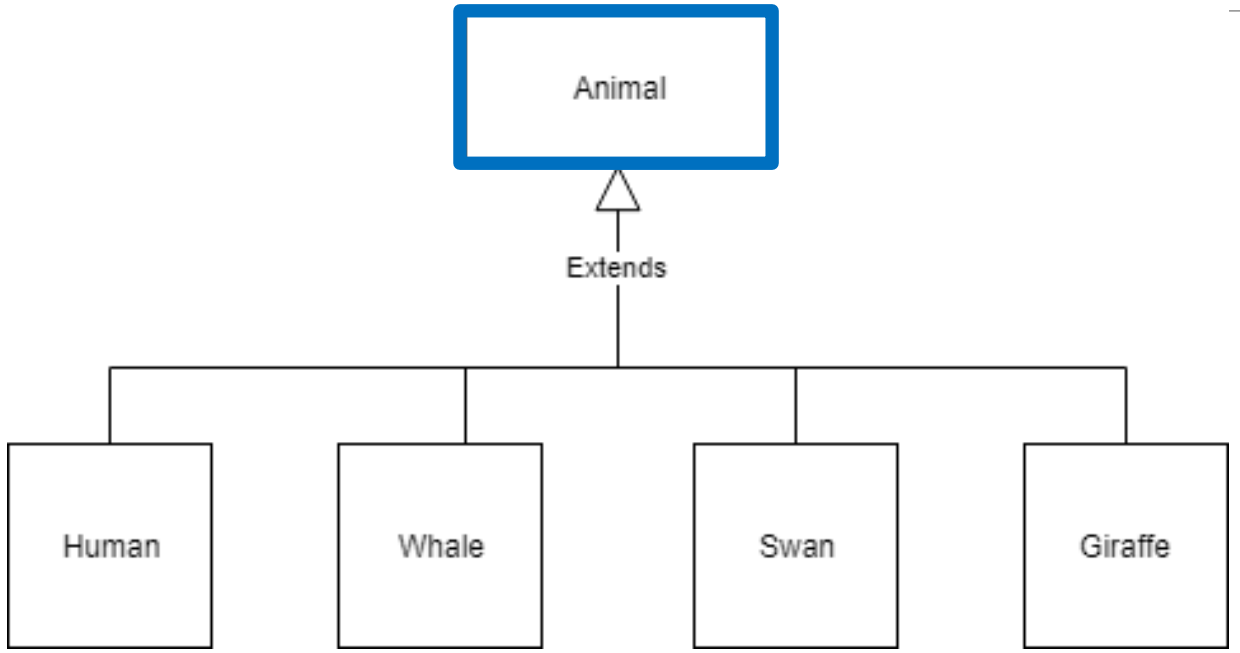
Now we just have a **widget list** filled with all the different kinds of **UI Widget objects**. (**UIPanel**, **UIScrollBar**, **UIButton** etc).

Since they all have the same interface (**draw method**) they can be treated as if they were their base class (UIWidget).

We don't care about the derived type. Let polymorphism determine the derived type at runtime

Interface segregation principle

Now Say We Were Modeling A Zoo…




```
class Animal(abc.ABC):

    def breathe(self):
        # common breathing code here

    def eat(self):
        # common eating code here

    @abc.abstractmethod
    def walk(self):
        # common walking code

    @abc.abstractmethod
    def swim(self):
        # common swimming code

    @abc.abstractmethod
    def fly(self):
        # common flying code
```

So we followed all our design principles and wrote the **Animal** ABC Class.

But this is terrible! **Now our Human class needs to override the fly method. Our Whale can walk!**

Interface Segregation Principle

Many client-specific interfaces are better than one general-purpose interface.

Or

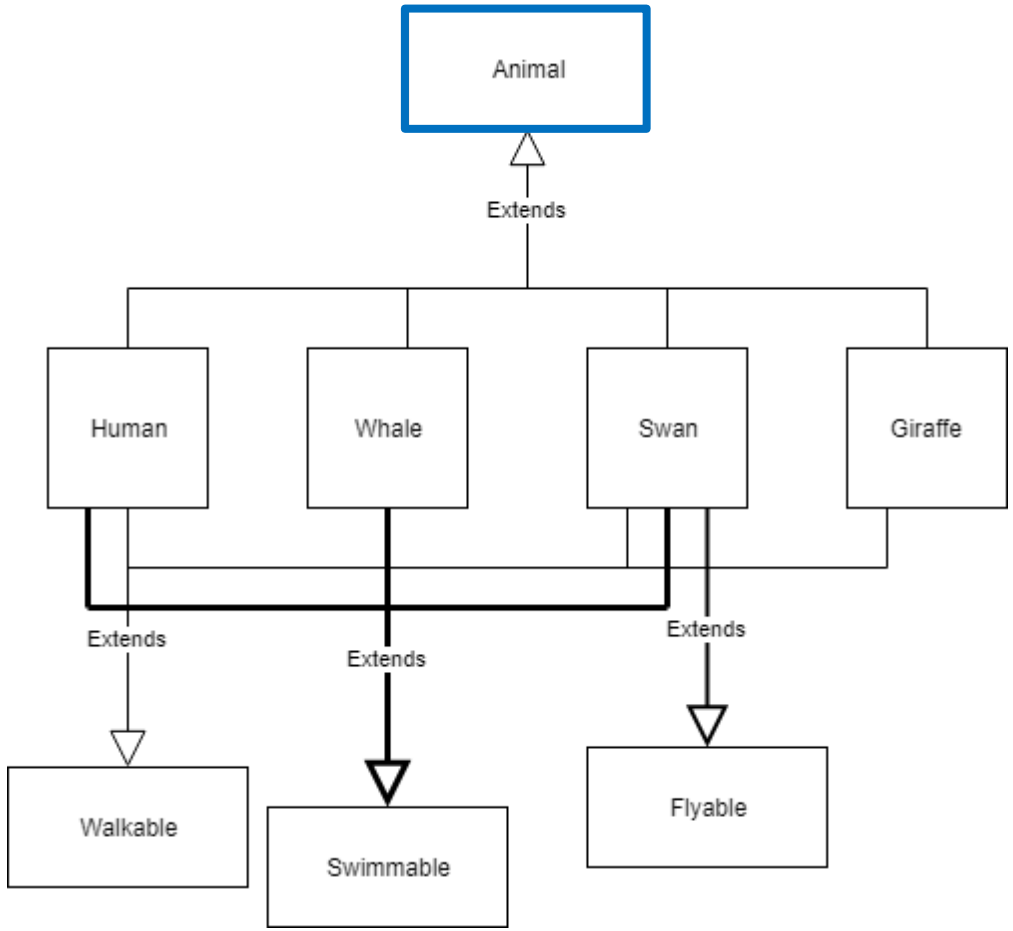
The interface segregation principle states that **no client should be forced to depend on methods it does not use.**

Put more simply: Do not add additional functionality to an existing interface by adding new methods.

This is the Single Responsibility Principle for Interfaces/Abstractions!

Back To Our Zoo!

Instead of polluting the **Animal** base class with additional functionality, we **create separate interfaces (read: ABC) that handle different responsibilities**.
This may seem difficult to maintain, but in fact it isn't.

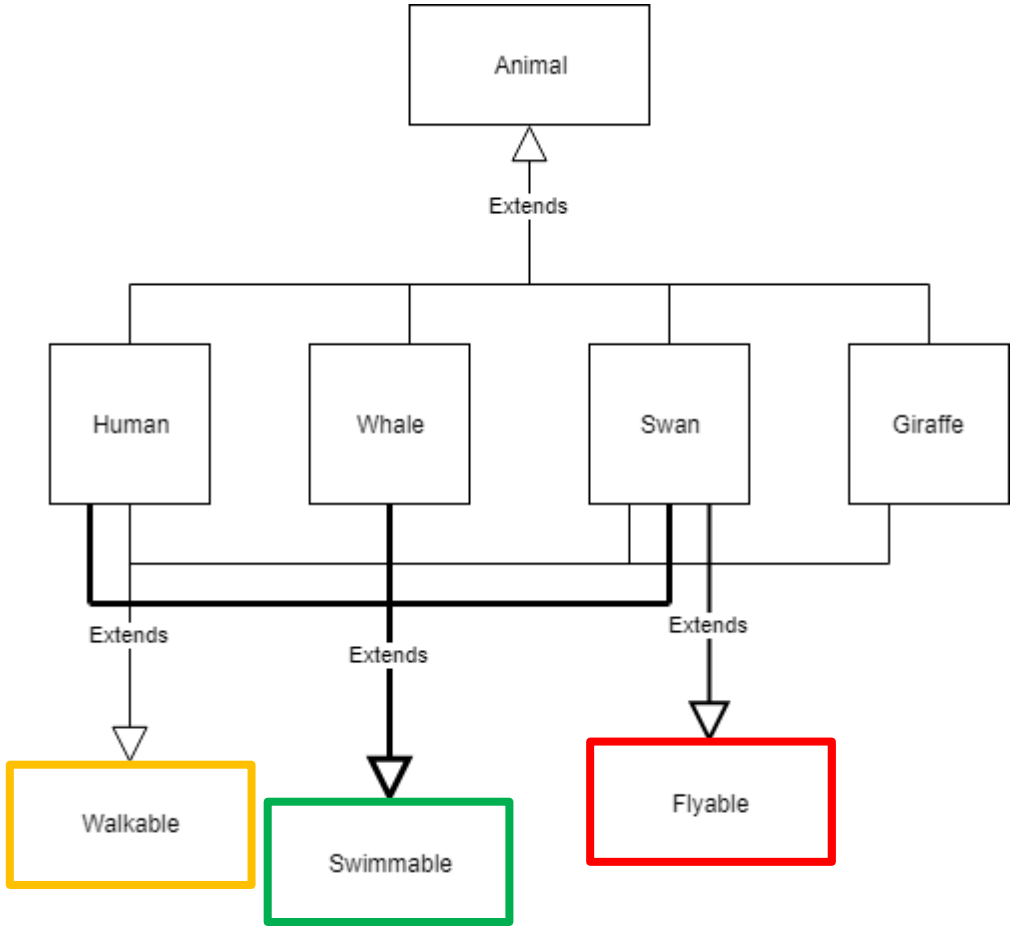


Extract code that **all Animals share**

```
class Animal:

    def breathe(self):
        #common breathing code here

    def eat(self):
        #common eating code here
```

Back To Our Zoo!

Instead of polluting the **Animal** base class with additional functionality, we **create separate interfaces (read: ABC) that handle different responsibilities**.
This may seem difficult to maintain, but in fact it isn't.

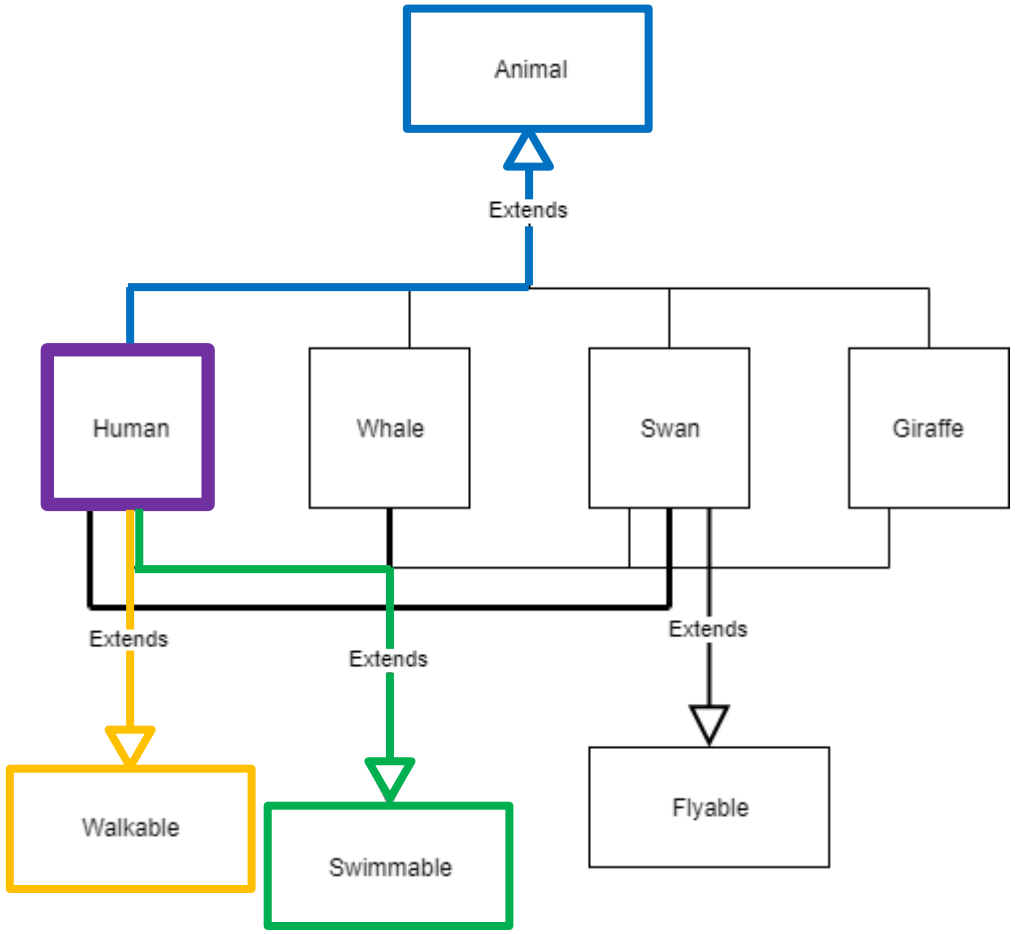


Extract optional functionality to **separate interfaces**

```
class Walkable(abc.ABC):
    @abc.abstractmethod
    def walk(self):
        pass

class Swimmable(abc.ABC):
    @abc.abstractmethod
    def swim(self):
        pass

class Flyable(abc.ABC):
    @abc.abstractmethod
    def fly(self):
        pass
```

Back To Our Zoo!

Instead of polluting the **Animal** base class with additional functionality, we **create separate interfaces (read: ABC) that handle different responsibilities**.
This may seem difficult to maintain, but in fact it isn't.



Human class inherits only the functionality they need

```
class Human(Animal, Walkable, Swimmable):
    def walk(self):
        # overridden walking code
    def swim(self):
        # overridden swimming code
```

Dependency inversion principle

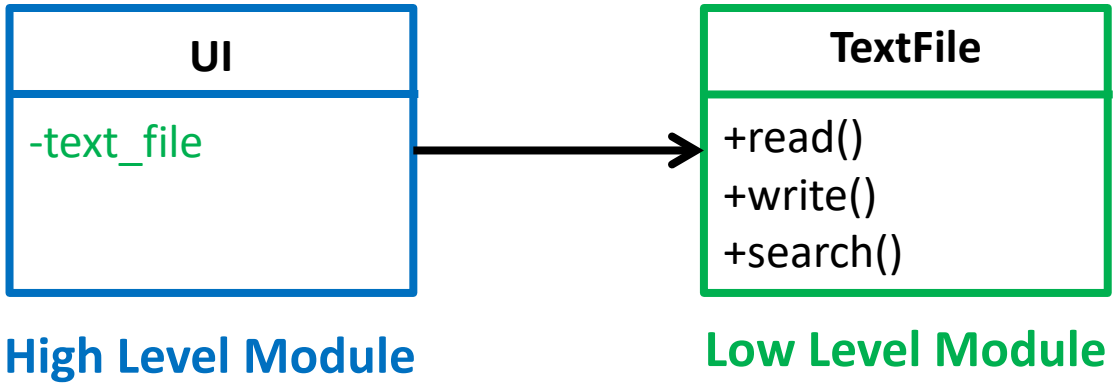
Dependency Inversion Principle

Dependency Inversion Principle can be thought as a combined effect of the previous principles.

Specifically, the **Open Closed Principle** and **Liskov Substitution Principle**

Before we dive into this, we need to look at a few concepts.

High Level & Low Level Modules

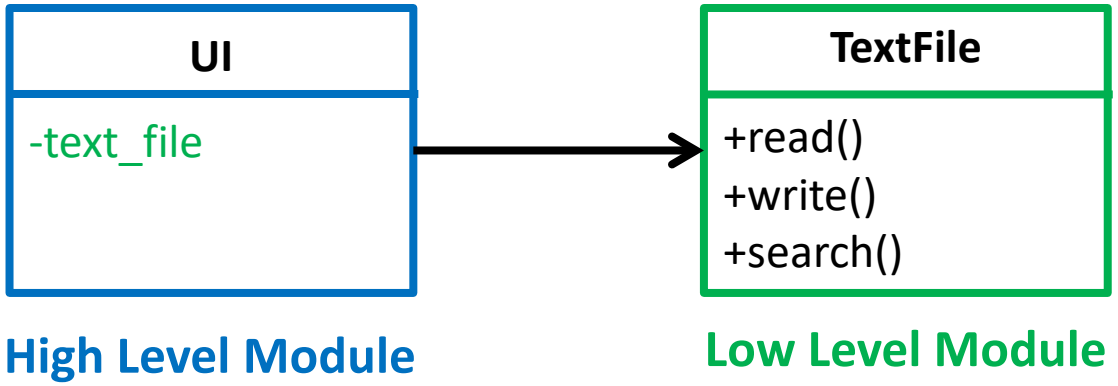


High Level Modules are the ones that contain complex logic made up of small 'chunks' of simpler code. For example, our UI class

A **Low Level Module** on the other hand, is a class that encapsulates some simple atomic behaviour. Such as writing and reading from a file, displaying an image, etc.

We generally compose high level modules with low level modules. That is, **high level modules are Coupled and Dependent on low level modules**

High Level & Low Level Modules



<u>This is not very good design.</u>

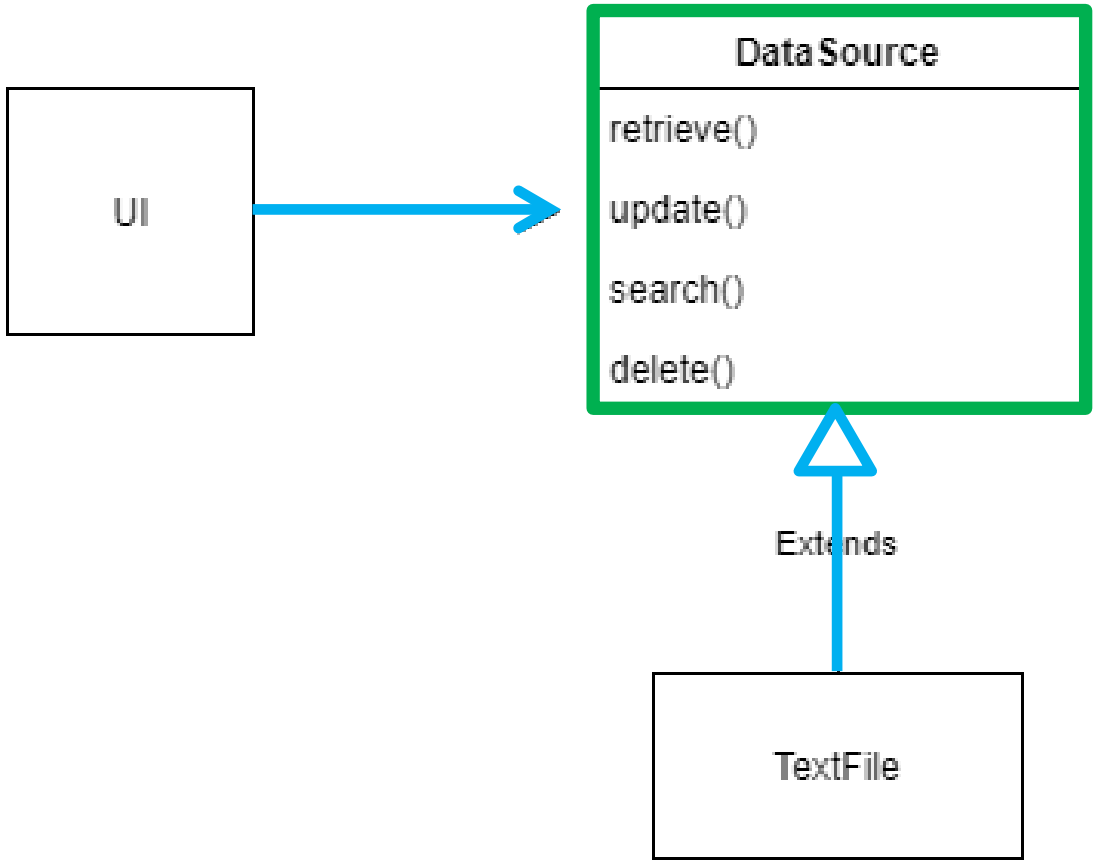
We don't want this dependency. We want to be able to change or replace the lower level modules without tampering with the complex code in the higher level modules

We inversed the dependencies!

We introduce an **abstract layer** that does not depend on anything.

We made both our **high level (UI)** and **low level (TextFile)** modules **depend on** the **abstract** instead.

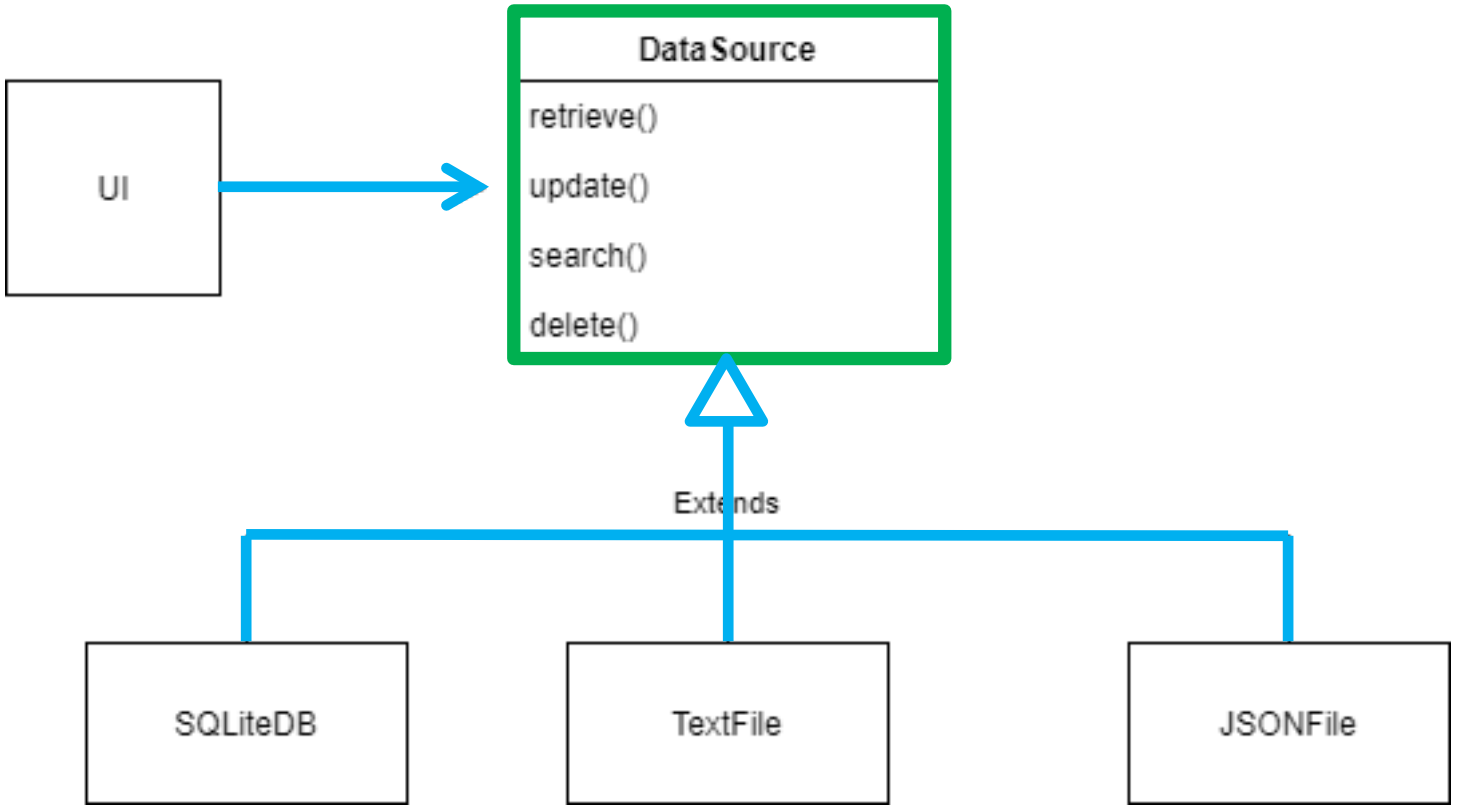
Now we can switch out either side, the **UI** and the **TextFile** could be replaced with something completely different



We inversed the dependencies!

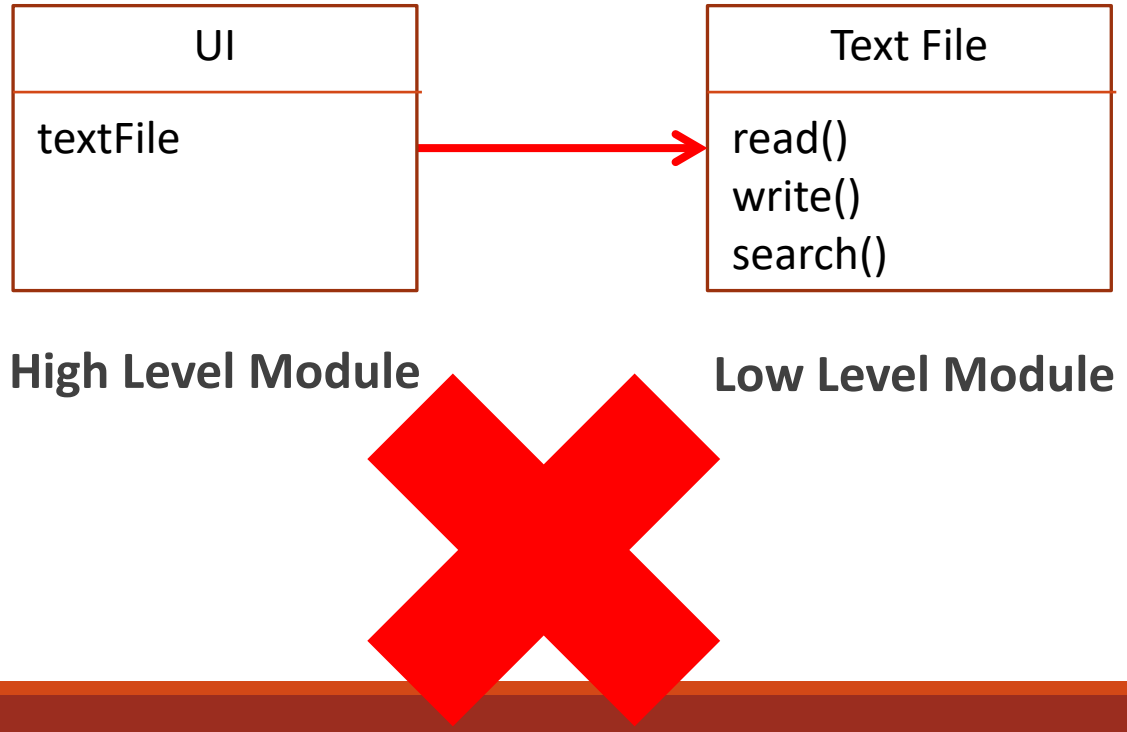
Everyone here has seen a diagram like this many times now.

We can even see the building blocks that are **Liskov Substitution** and **Open Closed Principles.**

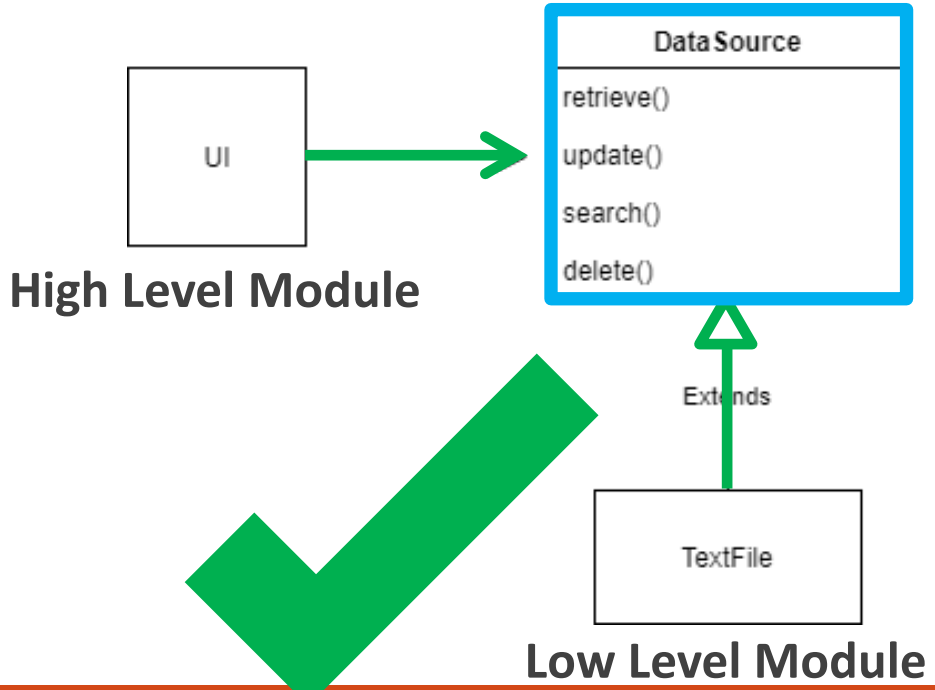


Dependency Inversion Principle

1. **High-level modules should not depend on low-level modules.**

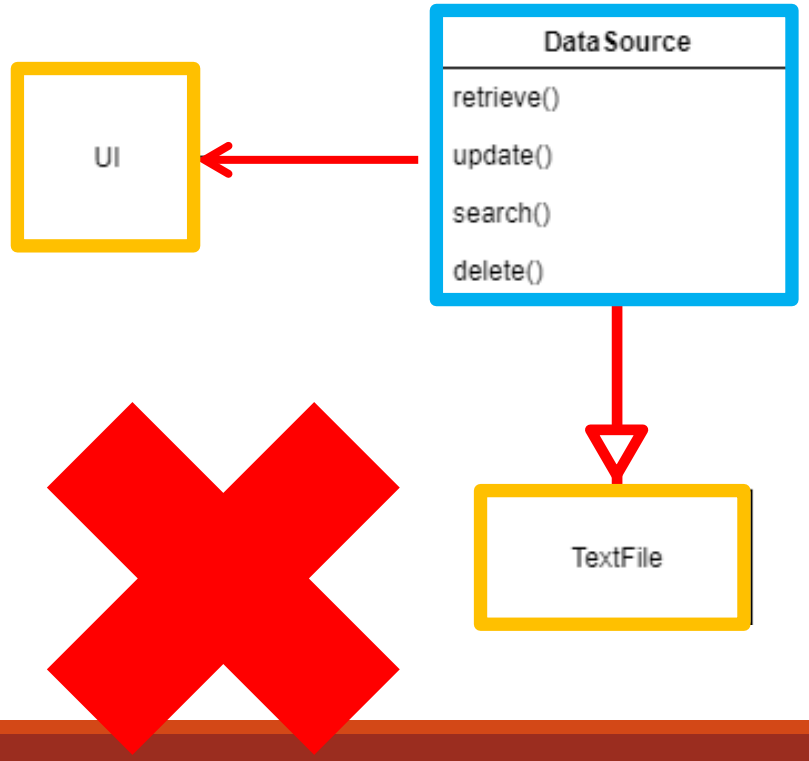


High-level modules and **low-level modules should depend on abstractions.**

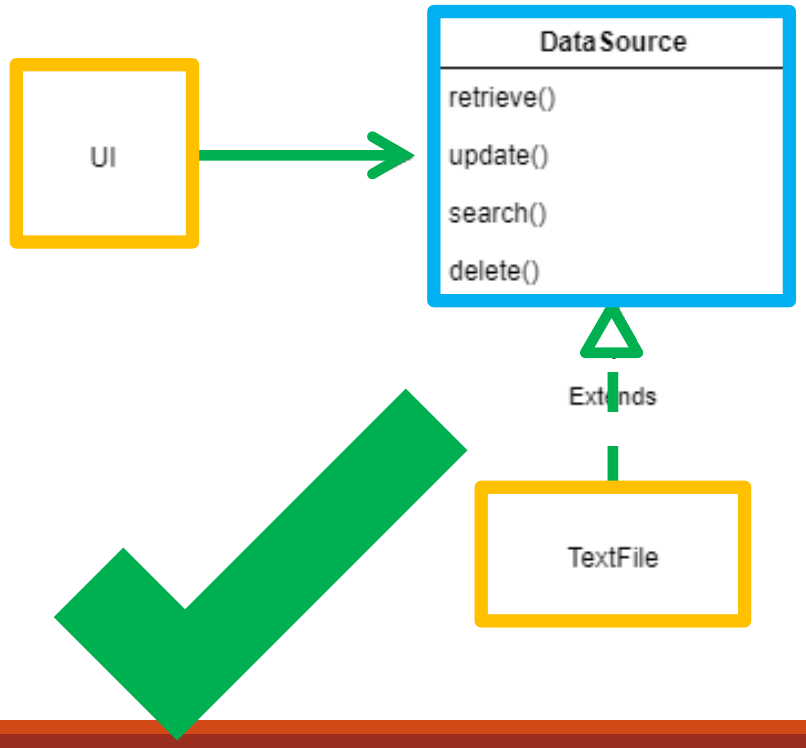


Dependency Inversion Principle

2. Abstractions should not depend on details



Details should depend on abstractions



Dependency Inversion Principle

In the example we just saw, the **DataSource** is our **abstract class or interface**, and the **UI** and **TextFile** classes are **concrete classes**.

Abstract classes

- declare an interface

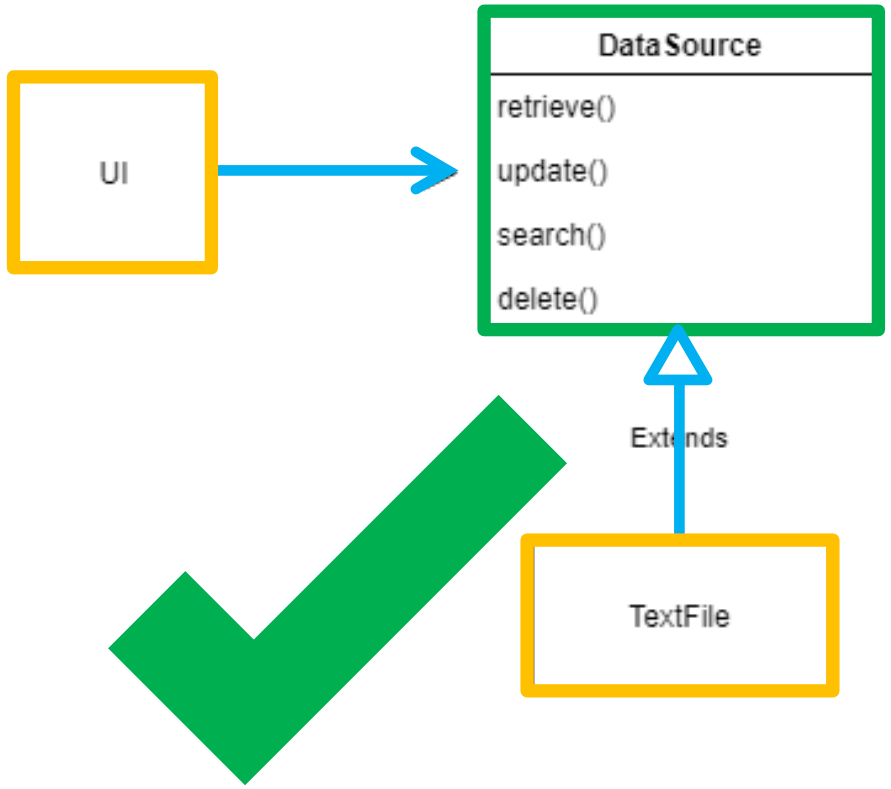
- can NOT be instantiated

Concrete classes

- contain the code that implements the details of the interface (**TextFile**)

- are fully implemented classes (**UI, TextFile**)

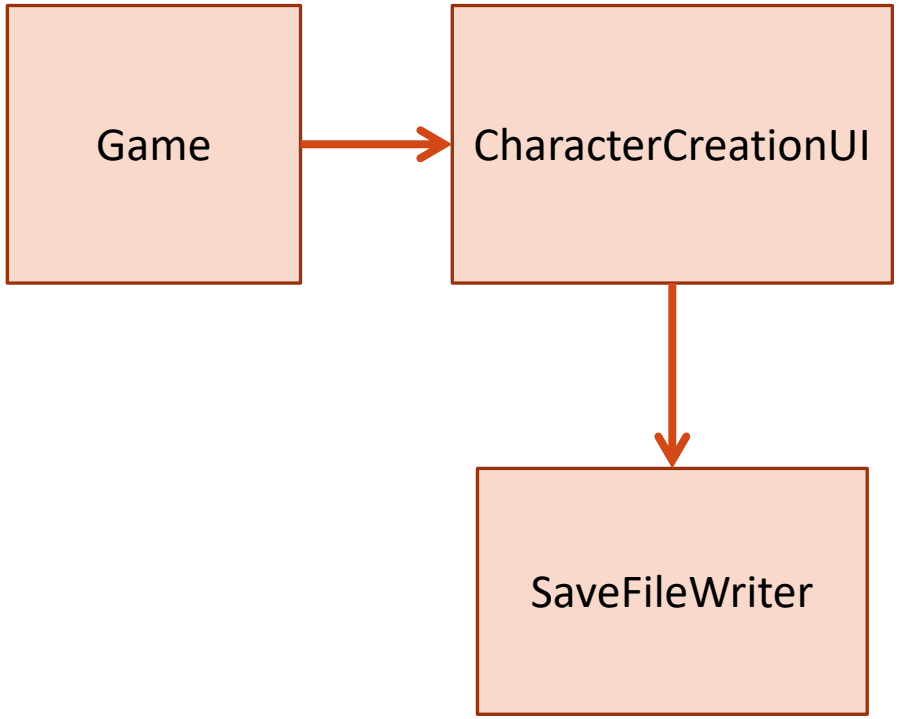
- CAN be instantiated



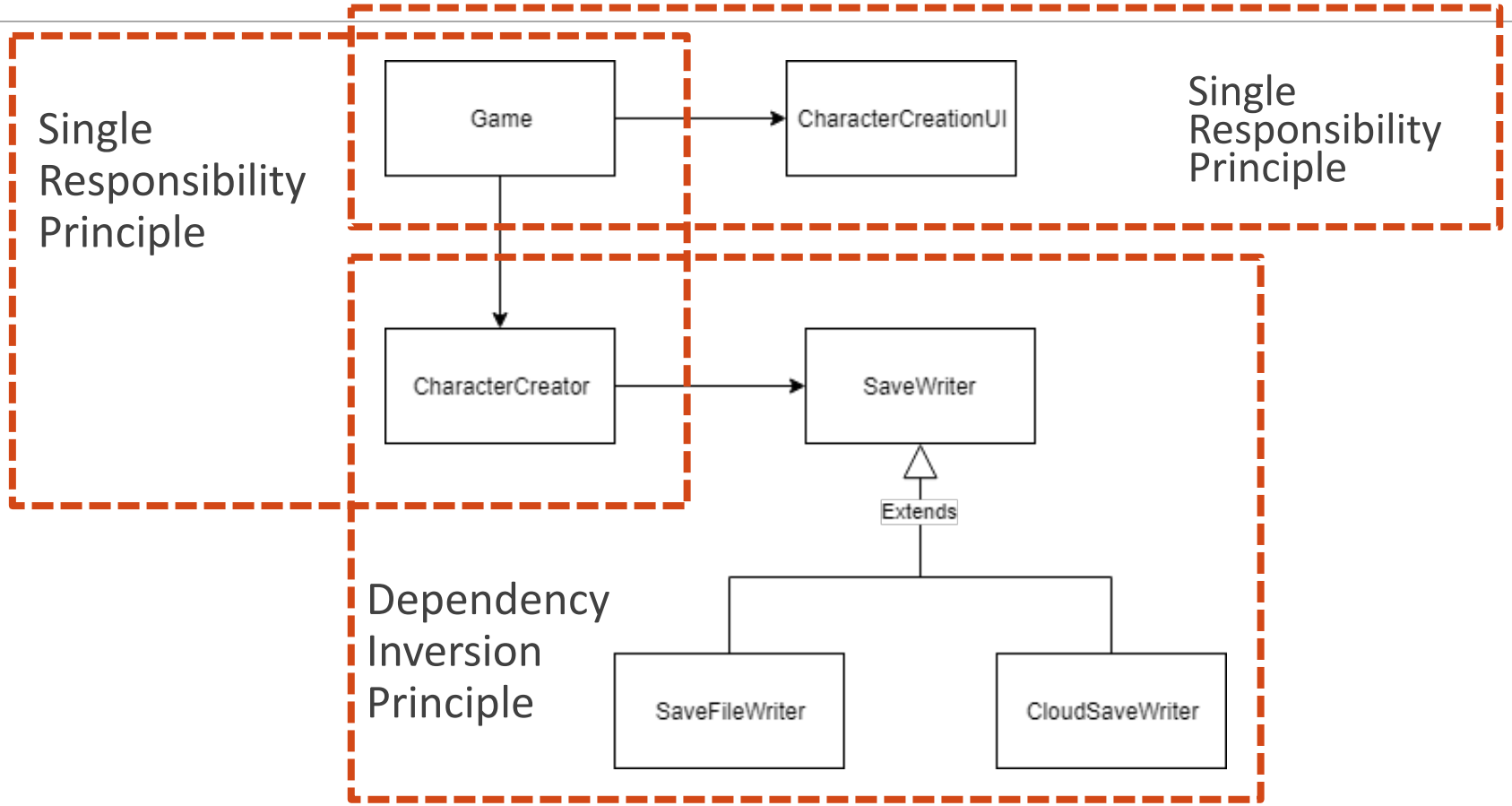
Right, now your turn! Say we had a character creation UI in a game.

We have a **game** with a **Character Creation UI**, where a player creates their character. When they click the create button, it gets saved to a **save file locally or online**.

Apply everything you've learnt so far to "fix" this terrible design.



My Solution


Single Responsibility Principle
Game
CharacterCreationUI
Single Responsibility Principle
CharacterCreator
SaveWriter
Extends
Dependency Inversion Principle
SaveFileWriter
CloudSaveWriter

Law of Demeter

Law of Demeter

Real life - Wallet example

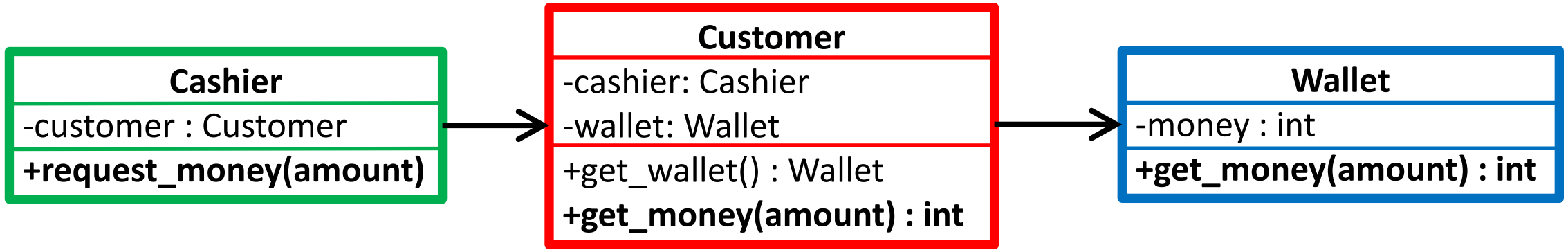
Law of Demeter

- Imagine if **Cashier**, **Customer**, and **Wallet** are classes

Goal: Cashier wants to get **int money** from the **Wallet**

- **Cashier** *should not* reach in **Customer**, then reach into their **Wallet** to get the money.
  - In **Cashier** **take_money()**: `money = self._customer.get_wallet().get_money(amount)` #BAD
- Classes **should not** have knowledge of public members several classes away

Law of Demeter



Goal: Cashier wants to get **int money** from the Wallet

- Cashier can access Customer, but should not access Wallet through Customer
- Refactor code so Cashier requests money from Customer. Customer accesses own Wallet to give money
  - In Cashier **request_money()**: `money = self._customer.get_money(amount) #OK!`
  - In Customer **get_money()**: `return self._wallet.get_money(amount) #OK!`
- Classes **should** have knowledge of public members of **neighboring classes**

Law of Demeter

"Each unit should have only limited knowledge about other units: only units "closely" related to the current unit. **Each unit should only talk to its friends**; don't talk to strangers."

Don't have train wrecks like these:

```
obj.getX().getY().getZ().doSomething();
```

Or

```
MyCharacterUI.GetCharacterData().CreateCharacter().Save()
```

Poor Demeter is often forgotten, some people even call it "Suggestion of Demeter" since this is difficult to avoid

Law of Demeter – Example Code

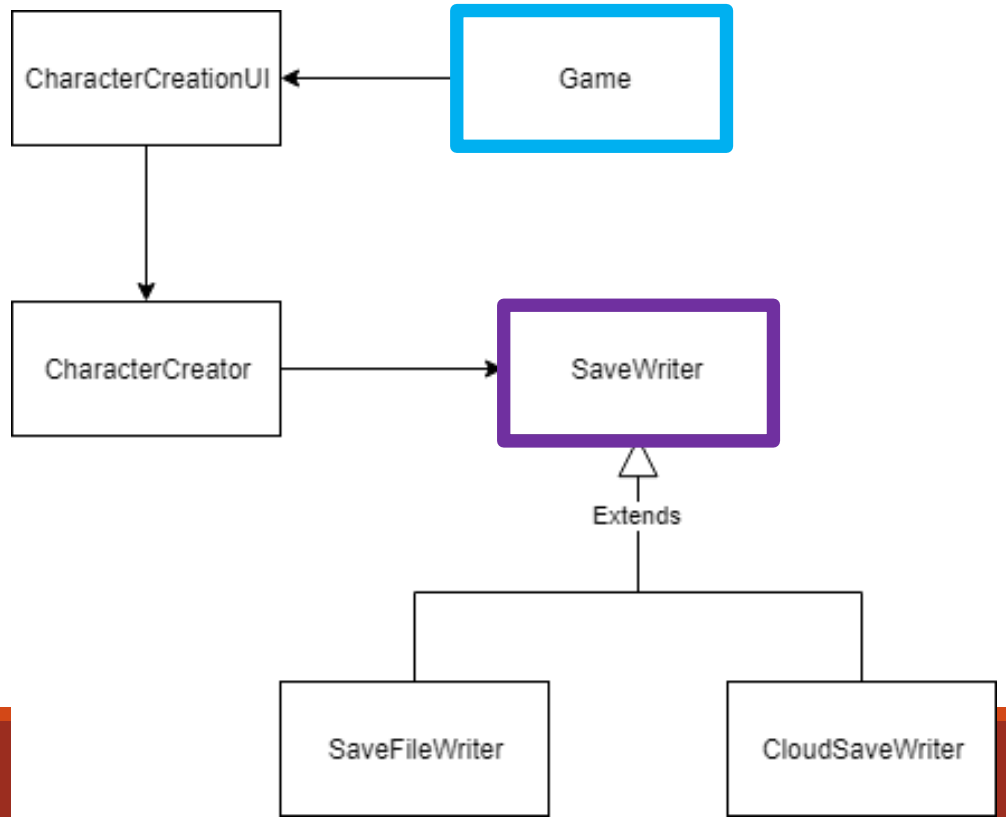
```
class Game:
    def create_character(self):
        self._data =
self.character_ui.get_character_creator().save(self.character_ui.get_character_creator().create_character())
```

This code tries to create a character and save it

This is obviously **bad** and **hard to read** code

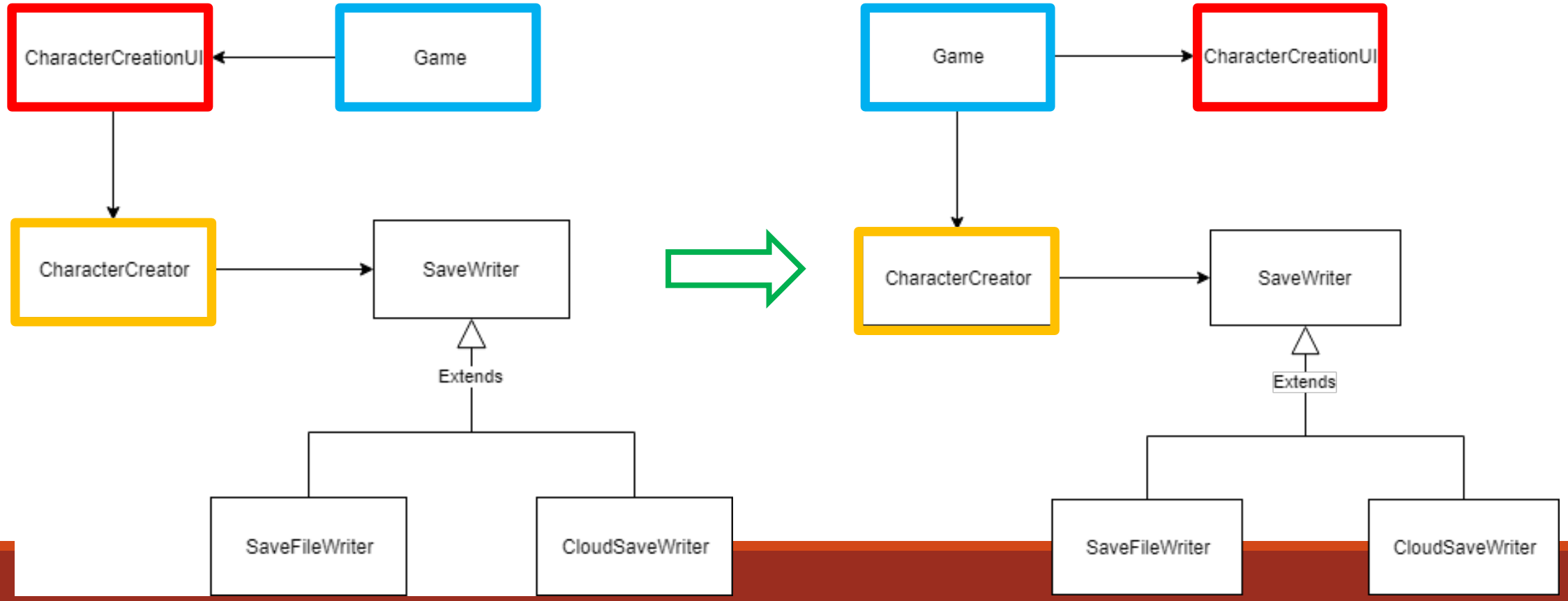
I also had a hard time getting the lines to fit in powerpoint. Bad for many reasons

Any change in the **save method**, which is all the way in **SaveWriter**, could have a ripple effect of changes all the way to our **Game** class

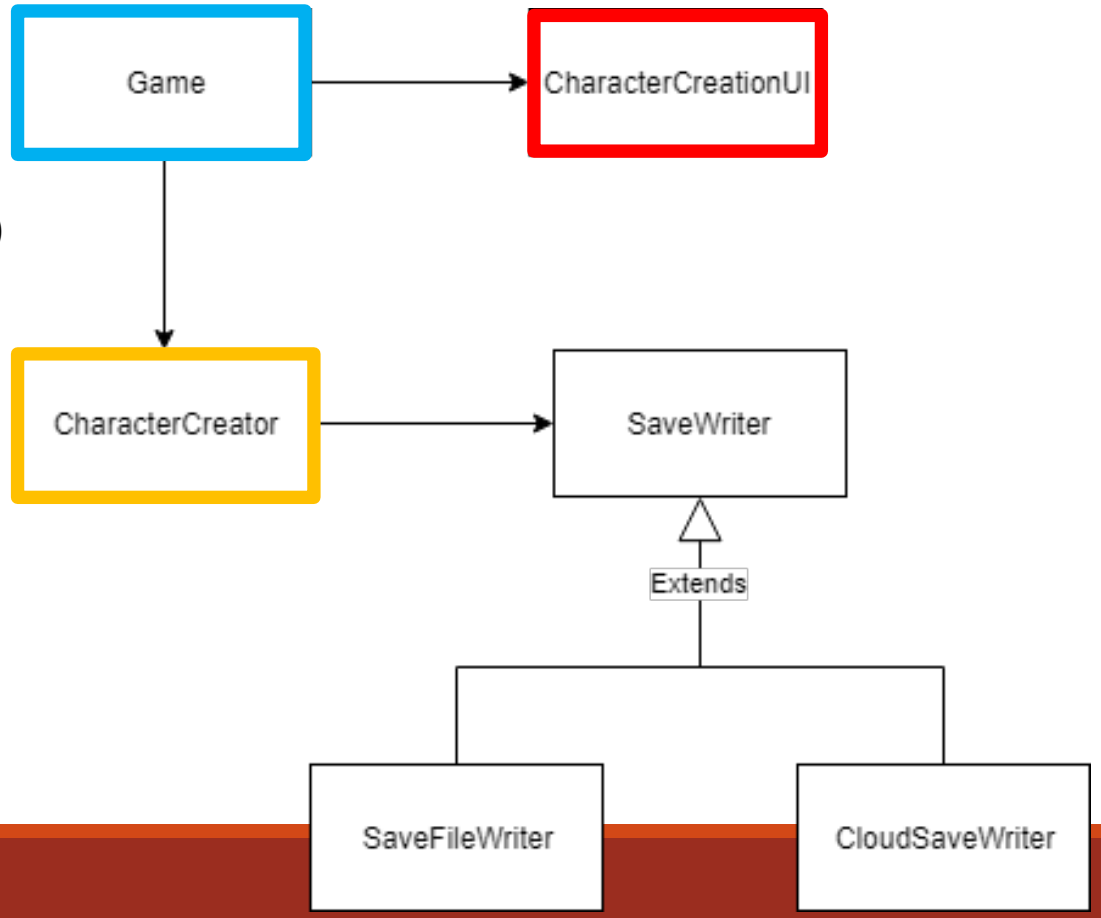


Law of Demeter – Example Code

- Part of the solution is to restructure the system by swapping **Game** and **CharacterCreationUI**
- Therefore **Game** does not need to *reach through* **CharacterCreationUI** to get **CharacterCreator**



Law of Demeter – Example Code - Solution

Restructure code so each class is responsible for their own data. This prevents classes reaching deeply through multiple classes

```
class Game:
    def create_character(self):
        data = self.character_ui.get_data()
        self.character_creator.initiate_character_creation(data)

class CharacterCreationUI:
    def get_data(self):
        return self.character_data

class CharacterCreator:
    def initiate_character_creation(self, data):
        my_char = self.create_character(data)
        self.save_writer.save(my_char)
```